



國立臺灣科技大學

資訊管理學系

碩士學位論文

學號：M10809308

支持零信賴架構的 OAuth 令牌身分鑑別機制

An enhanced token-based authentication
scheme using OAuth for Zero Trust
Architecture

研 究 生：黃婉筑

指導教授：查士朝 博士

中 華 民 國 一 一 〇 年 七 月



碩士學位論文指導教授推薦書

Master's Thesis Recommendation Form



M10809308

系所：

資訊管理系

Department/Graduate Institute

Department of Information Management

姓名：

黃婉筑

Name

HUANG, WAN-CHU

論文題目：

支持零信賴架構的OAuth令牌身分鑑別機制

(Thesis Title)

An enhanced token-based authentication scheme using OAuth for Zero Trust Architecture

係由本人指導撰述，同意提付審查。

This is to certify that the thesis submitted by the student named above, has been written under my supervision. I hereby approve this thesis to be applied for examination.

指導教授簽章：

Advisor's Signature

查士軒

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

日期：

Date(yyyy/mm/dd)

2021 / 7 / 27



碩士學位考試委員審定書

Qualification Form by Master's Degree Examination Committee



M10809308

系所：資訊管理系
Department/Graduate Institute Department of Information Management

姓名：黃婉筑
Name HUANG, WAN-CHU

論文題目：支持零信賴架構的OAuth令牌身分鑑別機制
(Thesis Title) An enhanced token-based authentication scheme using OAuth for Zero Trust Architecture

經本委員會審定通過，特此證明。

This is to certify that the thesis submitted by the student named above, is qualified and approved by the Examination Committee.

學位考試委員會

Degree Examination Committee

委員簽章：

Member's Signatures

羅乃維

葉國輝

查士朝

指導教授簽章：

Advisor's Signature

查士朝

共同指導教授簽章（如有）：

Co-advisor's Signature (if any)

系所（學程）主任（所長）簽章：

Department/Study Program/Graduate Institute Chair's Signature

羅乃維

日期：

Date(yyyy/mm/dd)

110/7/27

摘要

目前應用程式為了提供使用者更貼近需求的服務，會需要取得使用者其他服務的資料，而目前一般會使用 OAuth 協定：OAuth 讓使用者同意授權後，給予要求個資服務的提供者取得使用者資料的令牌。雖然透過令牌方式鑑別身分是一個非常快速又簡便的方法，但不論是誰只要能夠獲取令牌就能拿到使用者的資料，因而帶來安全風險。這樣的做法也被應用在企業網路內，做為單一登入的基礎。但如今因遠距辦公興起，傳統企業網路邊界模糊化，攻擊者只要能突破邊界，就能夠進行後續橫向移動，因此我們需要解決過去只重視邊界的問題，讓傳統以令牌為基礎的鑑別方式在現今邊界模糊的情況下，也能清楚確認客戶端身分，並給予敏感資料。

本研究提出的方法主要是基於零信賴架構原則，零信賴架構是針對傳統強化邊界的方法無法因應現今存取方式改變的問題，而要求不可以是否處於企業內網作為存取控制的依據。不論是誰，只要需要進入內網都需要做身分識別，而每次連線也都會依據此次連線的狀態做權限更動。有鑑於此，本研究提出的框架會在初次使用時做設備綁定，在後續進行授權或提出要求時做身分驗證，確保提出要求的使用者與拿取令牌的使用者為同一人，同時也會透過每次連線的設備狀況做風險評估來進行後續資料授權事宜，相較於傳統透過令牌做身分驗證可能造成的資料外洩，本研究貢獻在於能夠確保使用令牌的使用者身分，同時又可基於使用者每次不同的設備狀況動態更改授權權限，減少資安風險。

關鍵字：OAuth、身分鑑別、零信賴架構

Abstract

The OAuth protocol is widely used for applications to obtain user data from other services to provide personalized services: with the OAuth protocol, an application obtains consent from the user and also obtains the access token for seamless access to user data in a service. Therefore, users do not need to provide data to different applications repeatedly. In this case, if the access token is obtained by malicious actors, they can use it to access the user's data from the service. The OAuth protocol is also used in organizations for authentication and SSO. However, given the recent work-from-home trend, perimeter security safeguards have become less useful, making it easier for malicious actors to cross organizational perimeters and intercept OAuth tokens and utilize them to easily access unauthorized data. We must therefore verify user identities as users use tokens to access data or services.

In this study we thus propose a scheme to enhance a OAuth-based protocol based on the concepts of zero trust. In such a zero trust architecture, systems do not assume the existence of perimeter security safeguards. Systems must check user identity and permissions every time the user wishes to access his/her data. Also, systems should adapt access control policies based on the risks of access requests. In the proposed scheme, security risk assessment tools are installed on user devices. When a user accesses a service with access tokens, the service identifies the device and obtains the risks of the devices, after which the service can determine the user privileges based on the device contexts. This study will hopefully contribute to propose a scheme to extend the OAuth specification toward complying the tenets of a zero trust architecture.

Keywords : OAuth, Identity Authentication, Zero Trust Architecture

致謝

在碩士的兩年生活中，我從課堂和計畫中學習到了很多東西，不論是學術知識或是實作上的專業技能都增進許多。此論文的撰寫受到許多人的幫忙與建議才得以完成，在此非常感謝所有老師與同學們。

首先最感謝的是指導教授查士朝老師，從進入實驗室開始就讓我獲得許多珍貴的學習機會，不僅通過國際證照也參與了全國比賽，在這些難能可貴的經驗中使我成長了許多。老師在百忙之中撥空指導我的論文，提出很多實用的修改方向及論文結構的建議，讓我了解我研究方向的不足與改進目標。

同時，我也要感謝實驗室的學長姐們，張郁婷(Cathy)、許絲晴、張詠淳、李卓勳、張力仁及王聖傑對於新興技術的教導與協助計畫，感謝利文韡學長抽空為我們舉辦讀書會，並給予初期論文方向的建議與討論，也謝謝 PRlab 的同學們，黃憶萱、王俊傑、黃婉綺一同與我討論論文以及在各方面的幫忙，另外也要感謝學弟妹們：劉洸昕、謝宜庭、郭家定、張岑軒(Orange)，在實驗室為我們帶來許多歡樂的時光，非常感謝大家這兩年的陪伴，讓我們的研究生涯如此充實。

最後要感謝我的家人們，雖然因為距離關係不常回家，但永遠會在我需要支持時鼓勵我，讓我可以無後顧之憂地完成碩士學業，非常謝謝我的家人。

黃婉筑 謹誌於

國立臺灣科技大學 資訊管理研究所

中華民國一一〇年七月

目錄

摘要	I
Abstract.....	II
致謝	III
目錄	IV
圖目錄.....	VI
表目錄.....	VIII
第一章 緒論	1
1.1 前言	1
1.2 研究動機與目的.....	2
1.3 論文架構.....	3
第二章 文獻探討	4
2.1 OAuth.....	4
2.2 OAuth Single Sign On.....	11
2.3 Open ID Connect.....	13
2.4 零信賴架構	15
第三章 系統需求分析與定義.....	26
3.1 問題定義.....	26
3.2 需求分析.....	28

3.3 情境架構.....	30
第四章 框架概述與服務流程.....	32
4.1 框架概述.....	32
4.2 服務流程.....	34
第五章 實作與分析.....	49
5.1 系統實作環境.....	49
5.2 系統框架實作概述	50
5.3 系統效能測試	53
5.4 系統安全性分析.....	58
第六章 結論與未來發展	66
參考文獻.....	67



圖目錄

圖 1 OAuth 協定基本流程	5
圖 2 授權代碼授予執行流程	7
圖 3 隱式授予執行流程	8
圖 4 資源擁有者密碼憑證授予執行流程	9
圖 5 客戶端憑證授予執行流程	10
圖 6 OAuth SSO 概念	11
圖 7 OAuth SSO 流程	12
圖 8 OpenID Connect 流程	13
圖 9 ZTA 核心元件圖	17
圖 10 基於設備代理部署模型	19
圖 11 基於限制區域部署模型	20
圖 12 資源門戶模型	21
圖 13 雲端部署模型	21
圖 14 Google BeyondCorp 組件	23
圖 15 OAuth SSO 情境	26
圖 16 OAuth SSO 攻擊者情境	27
圖 17 使用情境	31
圖 18 系統框架	33

圖 19 框架作業流程圖.....	35
圖 20 框架步驟圖	36
圖 21 初始階段流程	40
圖 22 存取要求階段流程	43
圖 23 註銷流程.....	48
圖 24 框架實作流程	50
圖 25 設備令牌產生及加密程式碼.....	51
圖 26 單一設備執行 100 次的綁定時間圖.....	54
圖 27 單一設備執行 100 次的驗證時間圖.....	55
圖 28 單一設備執行 100 次的評估風險時間圖	56
圖 29 兩者方法差異時間比較.....	57

表目錄

表 1 OAuth 四種流程比較	6
表 2 不同部署模式的比較.....	18
表 3 框架角色定義.....	33
表 4 使用參數	38
表 5 設備令牌參數.....	39
表 6 判斷設備的安全性的依據.....	46
表 7 最終給予受保護資料等級比較表.....	47
表 8 設備綁定與驗證格式.....	51
表 9 設備風險評估與授權資料格式	51
表 10 方法比較.....	52
表 11 設備綁定 API 資訊	54
表 12 設備驗證 API 資訊	55
表 13 評估風險 API 資訊	56
表 14 圖 20 中各步驟資料傳輸說明及攻擊手法探討	60

第一章 緒論

1.1 前言

近年來，在科技迅速發展之下，眾多社群服務或雲端服務皆會使用"令牌(Token)"來作為使用者的身分識別及存取權限的控制，目前許多服務皆會使用 Token 來控管使用者的使用權限，例如用 Token 來作為門禁，來識別使用者是否有權進入。而為了讓使用者在使用各軟體或服務上有更好的便利性，各服務藉由使用者的授權來互相傳遞訊息，提供使用者更客製化的服務，其中就是藉由 Token 的傳遞來作為使用者授權的認證，佐證使用者已同意授權並且授權哪些範圍。

透過使用者授權第三方應用程式能獲取自身敏感資料的整體流程可參考 OAuth2.0，網際網路工程任務組(Internet Engineering Task Force, IETF)於 2012 年提出 The OAuth 2.0 Authorization Framework[1]，主要是在訂定一個開放授權框架，讓使用者在不洩漏自身帳號密碼的情況下，授權自己受保護的個人資料給第三方應用程式使用，整體來說，透過使用存取令牌(Access Token)來作為驗證的因素。目前使用 OAuth2.0 授權框架的服務非常多，像是 Open Banking[2]或 Blue Button[3]等，但 OAuth2.0 主要是針對身分「授權」訂定協議及框架，對於使用者或第三方應用程式的身分認證上，OAuth2.0 無特別強調。

以往企業常使用 OAuth2.0 協定來進行身分驗證。這樣的作法無法因應現今的存取需求：現今越來越多遠距辦公、外部存取等需求，遠距工作的連線模式可包含遠端登入、遠端桌面或透過雲端等，設備有可能是公司發行的或者使用者私人設備等[4]，當有攻擊者竊取 Token，就可輕易拿取使用者敏感資料，像 Facebook 於 2018 年就有發生 Token 外洩[5]，也因為遠距工作的發展，使用者的位置遍及各處，無法用以往透過邊界來判定使用者是否可信的方式來評估使用者，因此我們需要強化傳統邊界，透過零信賴這種持續驗證且不輕易信賴任何人的原則來強化傳統使用 Token-based Authentication 的方式，跳脫以往使用 Token-based 的驗證思維。

1.2 研究動機與目的

本研究針對美國國家標準暨技術研究院(National Institute of Standards and Technology, NIST)於近年提出零信賴架構(Zero Trust Architecture, ZTA)[6]，將其與 OAuth 架構作比較，提出一個使 OAuth 應用支援零信賴架構原則的方法。因應 ZTA 主要是不信賴任何設備或人員，不論要求的對象是否來自企業內網或外在的不信賴環境，都需一視同仁的將其預設為可能攻擊對象，而過往企業內部使用者要存取時都會使用 OAuth 或 OAuth SSO，但因為辦公模式改變，使用者如在家工作，會需要從遠端存取企業資源，此時企業若像以往僅信任令牌的話，容易導致資安風險，因此本研究透過將 ZTA 原則與 OAuth 比較，來強化過去只重視邊界的問題。當我們將 ZTA 原則套用到 OAuth 上時，會發現有幾點不足的地方，如：無法驗證客戶端身分、無法針對每筆交易做驗證、無針對每筆交易做風險管理...等。本研究提出一個框架能夠將 OAuth 套用至 ZTA 環境上，解決過去客戶端認證機制不足，而有可能存取令牌被偷走的問題，確保使用令牌的一定是客戶端，而非攻擊者，在改動 OAuth 協定的最小變動原則下，讓使用者及客戶端都可放心使用 OAuth 來授權。

從 OAuth2.0 提出之後，許多企業或第三方應用程式都參考此規範建立自己的身分授權框架，同時在眾多服務的提供下，為了方便使用者能夠不需牢記多組帳號密碼，因而提出 OAuth Single Sign On(OAuth SSO)的概念，使用者能夠在登入後，同時取得所有系統的存取權限，不需要針對單一系統逐一登入[7]，雖然 OAuth SSO 已經是相當成熟的概念，但 OAuth SSO 流程中僅透過令牌做身分鑑別，且令牌無法精確驗證使用者身分，同時也無法動態判斷使用者裝置風險並決定授權權限，這樣無疑會增加許多資安風險。在如今新環境需要以零信賴架構為主的情況下，需要針對身分認證重新設計，將 token-based 方法強化，

本研究為了能避免上述資安問題，在基於 OAuth 協定中加入裝置鑑別機制，讓使用者與身分提供者會先進行設備綁定，在使用者要求存取應用程式時，身分提供者基於 OAuth 協定來協助應用程式進行裝置確認服務，而後再讓應用程式像存取保護伺服器確認使用者存取權限。透過上述的框架，不需要透過存取代理也可以讓 OAuth 支援零信賴架構原則內的每次連線要確認身分及依據每次狀況更改授權權限。

1.3 論文架構

本研究共分為六章節，各章節介紹內容如下：

第一章 緒論

主要闡述研究背景、研究動機，主要說明目前 OAuth 應用所欠缺的安全考量，利用近期 NIST 提出的零信賴架構讓 OAuth 架構更安全，同時也提出研究目的與貢獻。

第二章 背景知識與相關文獻

針對 OAuth2.0 授權框架進行討論，了解 OAuth 標準會面臨的問題，也介紹 ZTA 相關標準，了解零信賴架構的模組與目前狀況。

第三章 系統需求分析與定義

本章節說明本研究的問題定義及需求分析，針對問題需求提出使 OAuth 應用支援零信賴架構原則的方法，並說明系統架構與流程。

第四章 框架概述與服務流程

詳細說明本研究所提出的方法，針對各組件與功能進行描述與討論。

第五章 實作與分析

針對本研究所提出的架構，進行效能測試，並且探討此框架的安全性。

第六章 結論與未來發展

總結本研究所所帶來的研究成果進行歸納與結論，並說明未來可延續或改善的方向。

第二章 文獻探討

本章節將介紹研究時所需的相關技術，並探討其相關背景知識，例如在研究 OAuth 過程中必需針對各授權框架作了解，才能更透徹的概覽這些相關技術，因此將會介紹 OAuth SSO 及 Open ID Connect 等，並透過參考零信賴架構提出讓 OAuth 能夠更為安全的做法。

2.1 OAuth

OAuth2.0 於 RFC6749 中定義授權框架，提供第三方應用程式在資源擁有者授權其訪問權限時，代表資源擁有者來獲取受限制的資源，同時該規範淘汰 RFC 5849 中所描述的 OAuth1.0 協議。因應在傳統的客戶端/伺服器驗證模型中，當客戶端需獲取被限制的資源時，需要使用擁有者的憑證來通過伺服器的驗證，因此使用者需與客戶端分享自身敏感的資訊，如：帳號密碼。

這樣的架構也會衍生出許多問題與限制：

- 第三方應用程式得以儲存使用資源擁有者的帳號密碼
- 資源擁有者無法控管第三方應用程式使用其憑證獲取那些訪問權限，同時也無法限制第三方訪問期限
- 當第三方案式被攻擊導致破解，使用者的機敏資訊即有外洩可能
- 資源擁有者無法撤銷單個第三方的訪問權限

OAuth2.0 提出此授權框架可以改善上述所說的問題，透過存取令牌作為依據，可取代以往使用資源擁有者帳號密碼作為授權的判斷，只要資源擁有者願意授權資料給第三方應用程式，伺服器就頒發存取令牌作為識別，同時此存取令牌會表示特定範圍、生存期限或訪問屬性等等來做為保護資源擁有者資料的依據。而 OAuth2.0 授權框架主要有四個角色：客戶端(client)、資源擁有者(Resource owner)、授權伺服器(Authorization Server)、資源伺服器(Resource Server)。

四個角色分別有不同的任務：

- 客戶端：第三方應用程式，需經過資源擁有者授權才能獲取資料
- 資源擁有者：擁有資料的使用者，能自我決定是否授權資料給客戶端

- 授權伺服器: 認證資源擁有者的身分，確認其是否同意授權給第三方，如願意授權則需核發存取令牌
- 資源伺服器: 存取使用者的資料，驗證客戶端所發送之存取需求即存取令牌，給予相對應的資源

OAuth 協定基本流程分為三大部分，如圖 1：

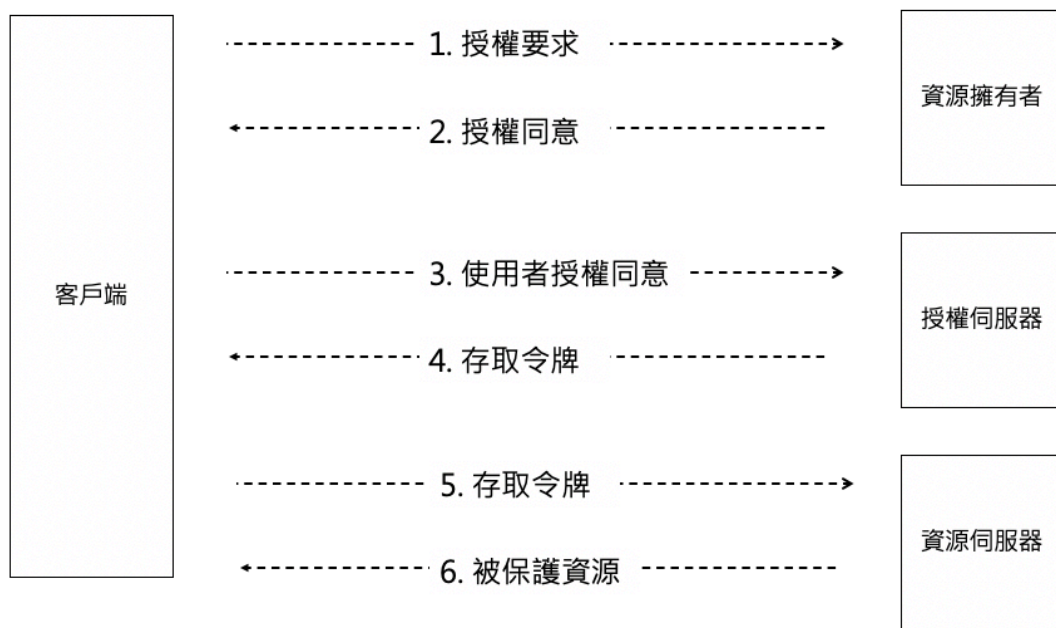


圖 1 OAuth 協定基本流程

(資料來源：整理自 Hardt, D.(2012)。The OAuth2.0 Authorization Framework)

1. 第三方應用程式先與資源擁有者要求授權
2. 在獲得資源擁有者授權後，跟授權伺服器換取存取令牌
3. 拿取存取令牌與資源伺服器換取資源擁有者受保護的資源

上述為 OAuth 基本協定流程，只要第三方跟使用者要求授權後，便可透過此流程來獲得其敏感資訊，除此之外，OAuth 標準中還定義了四種流程，分別為：授權代碼授予流程(Authorization Code Grant Flow)、隱式授予流程(Implicit Grant Flow)、資源擁有者密碼憑證授予流程(Resource Owner Password Credentials Grant Flow)、客戶端憑證授予流程(Client Credentials Grant Flow)[1]。

這四種流程皆不一樣，也分別對應不同的適用情境，我們可以看到表 1[9]來了解四種流程差別。

表 1 OAuth 四種流程比較

	授權代碼授予流程	隱式授予流程	資源擁有者密碼憑證授予流程	客戶端憑證授予流程
適用情境	安全性最高，最普遍被使用的流程	對安全要求不高情境	雙方需要高度信任	客戶端可自行請求訪問令牌
客戶端需提前至伺服器註冊	V	V	V	V
授權伺服器驗證使用者身分	V	V	V	
使用者同意授權客戶端	V	V	直接將使用者帳號密碼給予客戶端	
授權伺服器發 grant code 到客戶端	V			
客戶端通過存取令牌要求資料	V	V	V	V
安全性	最安全	存取令牌有外洩風險	使用者須將帳號密碼給客戶端，風險較高	客戶端直接使用其憑證進行驗證
實作難易度	流程最複雜，難易度最高	難易度不高，僅需透過轉址獲取存取令牌	難易度不高，驗證使用者身分後就可獲取存取令牌	難易度不高，僅需進行客戶端身分識別

(資料來源：整理自 oauthio (2018)。OAuth2 Introduction Through Flow Diagrams)

2.1.1 授權代碼授予流程

授權代碼授予基本情境為客戶端先跟授權伺服器要求授權代碼，待驗證後，就可透過此代碼要求存取令牌，而圖 2 則呈現授權代碼授予整體執行流程，客戶端需能與資源所有者的用戶代理進行溝通，同時能夠透過重定向來接收授權伺服器傳入的請求。(1)一開始客戶端通過將用戶代理定向到授權伺服器來啟動流程，(2)這時授權伺服器會透過使用者代理來驗證資源擁有者身分，同時也確定資源擁有者是否同意或拒絕授權客戶端的授權請求，(3)假設資源擁有者同意授權，授權伺服器便會重定向到客戶端，同時提供授權代碼。(4)之後客戶端可透過授權代碼來跟授權伺服器換取存取令牌，(5)授權伺服器會先驗證授權代碼，確保重定向 URI 與以往重導到客戶端的一樣，驗證過後，就會回傳存取令牌，(6)客戶端可透過此令牌向資源伺服器要求使用者資源。

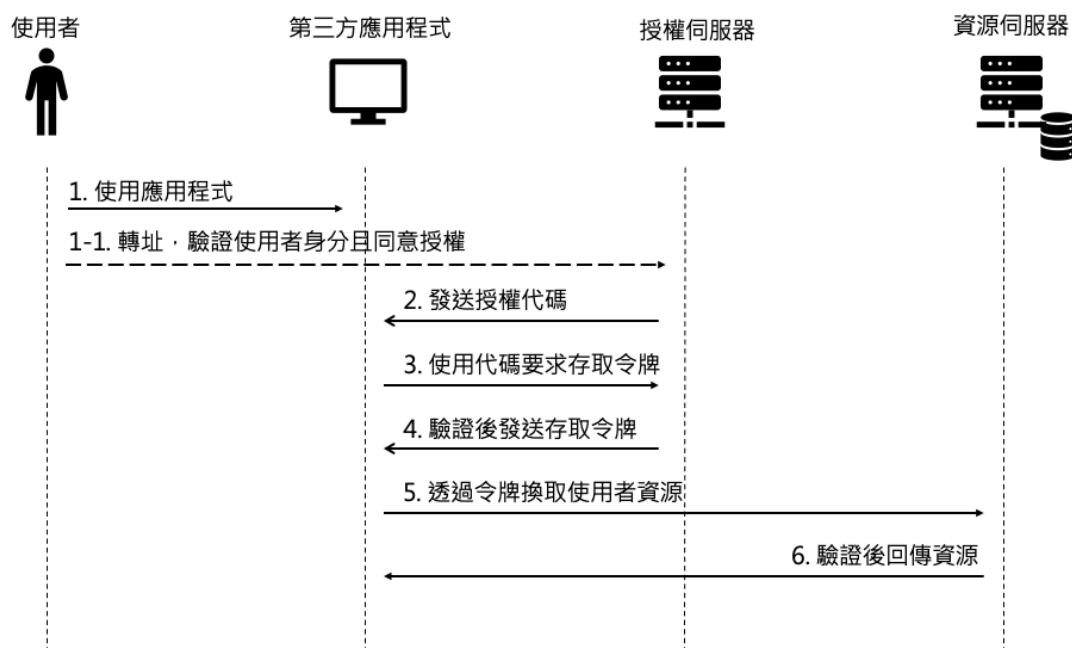


圖 2 授權代碼授予執行流程
(資料來源：本研究整理)

2.1.2 隱式授予流程

此流程的客戶端通常是在瀏覽器中使用腳本語言實現的，客戶端一樣需要能夠與資源所有者的用戶代理進行溝通，同時能接收來自授權伺服器傳入的請求。與授權代碼授予流程不同，在第一種流程中，客戶端會分別提出「授權請求」與

「存取令牌請求」，並且在授權請求中獲取授權代碼再透過此代碼去換取存取令牌；而在隱式流程中會直接提出「存取令牌請求」，當授權伺服器驗證過後會直接核發存取令牌並重導到轉址 URI 的位置，而存取令牌及相關資訊會透過#分段來傳遞，當資源永久者的用戶代理收到後，會提取分端中的存取令牌，再用此令牌像資源伺服器進行請求。因為不是透過後端來獲取存取令牌，因此會有安全疑慮，此令牌會有有暴露的可能性，可參考圖 3。

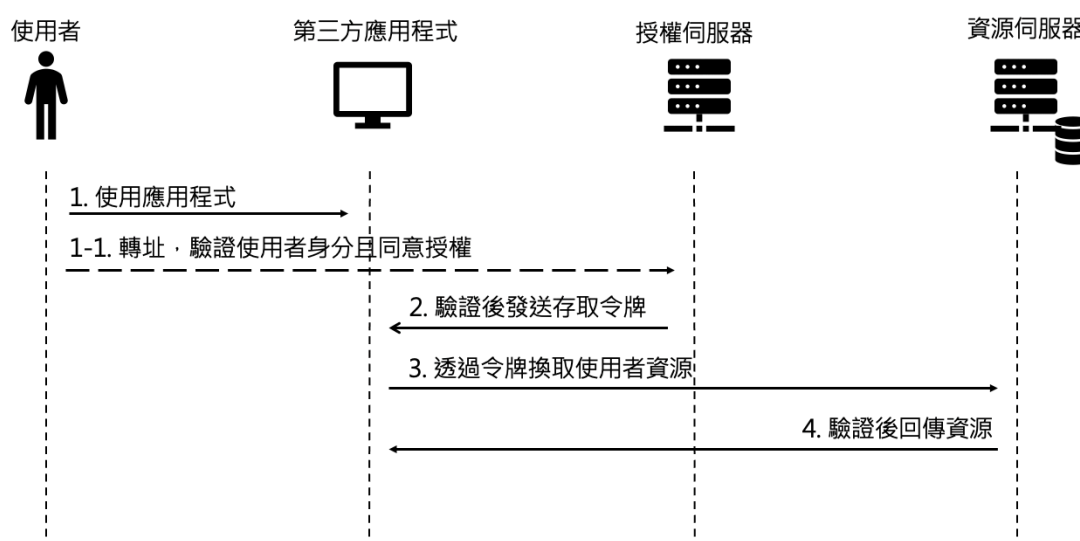


圖 3 隱式授予執行流程
(資料來源：本研究整理)

2.1.3 資源擁有者密碼憑證授予流程

基本上此授予流程需要高度的信賴關係，因資源擁有者要授權第三方應用程式獲取本身的資源，就需要將自己的憑證，也就是帳號密碼給第三方，如圖 4 表示，當資源擁有者對客戶端有極高的信賴，像設備操作系統或高特權應用，就可將自身的憑證給予第三方，而關於第三芳如何獲取此憑證不在 OAuth 規範考慮內。當第三方拿到憑證後，就可利用此憑證跟授權伺服器進行驗證，驗證成功後，一樣可獲得存取令牌，接下來與其他流程一樣，透過此存取令牌跟資源伺服器換取使用者資源。另外要特別注意的是，授權伺服器要在環境不允許其他流程的情況下才可啟用此授予類型。

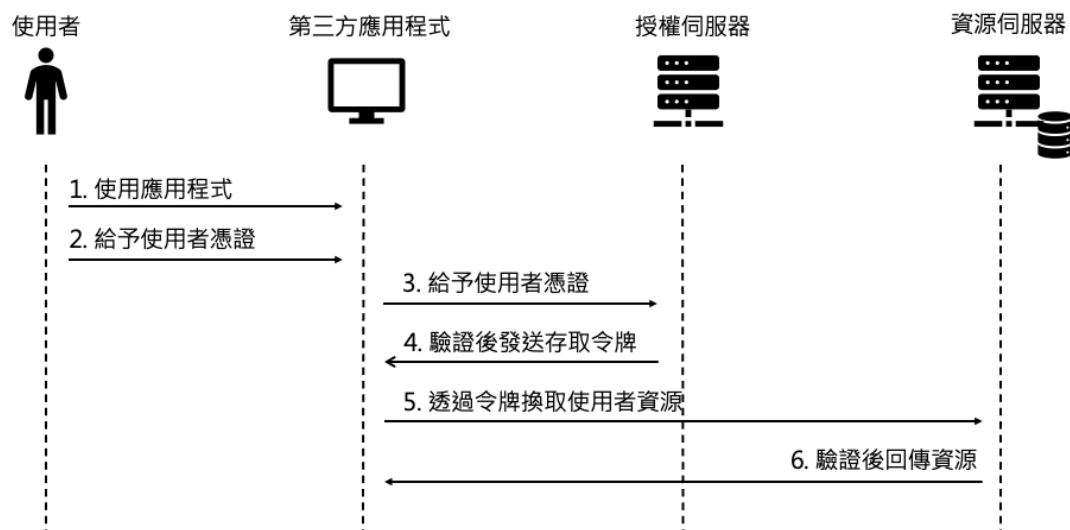


圖 4 資源擁有者密碼憑證授予執行流程
(資料來源：本研究整理)

2.1.4 客戶端憑證授予授予流程

當客戶端請求自身的受保護資源，也就是客戶端本身就是資源擁有者，或先前曾與授權伺服器安排擁有其他的資源所有者的存取權限，客戶端就可使用自身的憑證或其他受支持的身分驗證方式來請求存取令牌。由於將客戶端憑證作為授權授予，因此不需要其他授權請求，整體流程為客戶端將自身憑證傳給授權伺服器驗證，當身分驗證成功後，授權伺服器會回傳存取令牌，客戶端就可使用此存取令牌跟資源伺服器要求用戶受保護的資源，如圖 5。

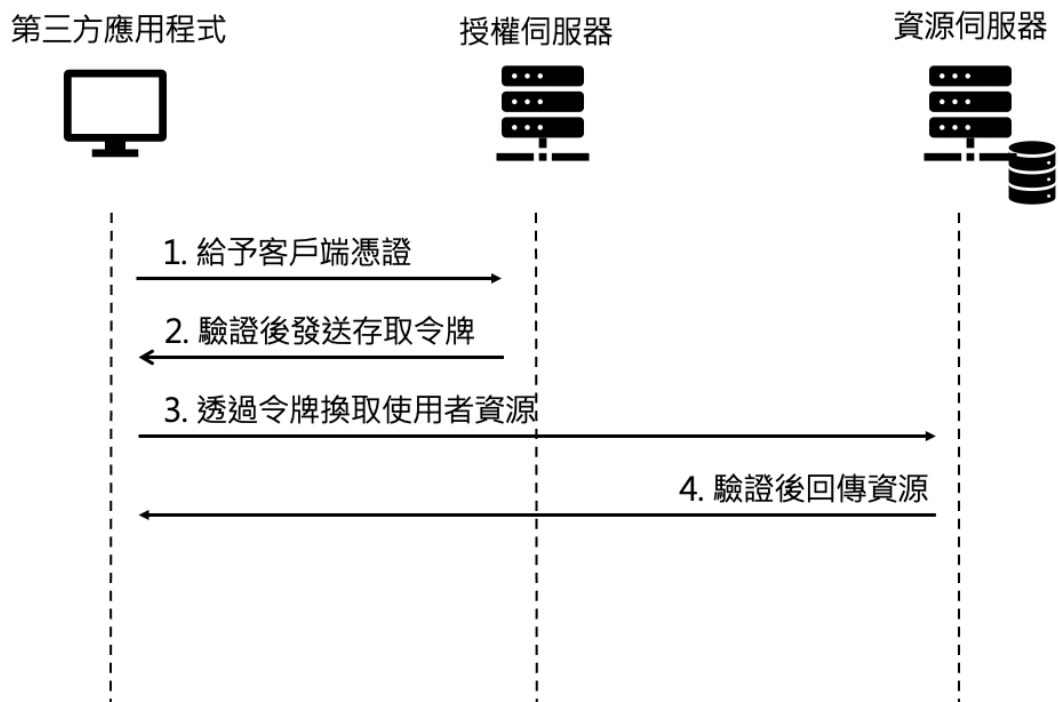


圖 5 客戶端憑證授予執行流程
(資料來源：本研究整理)

四種授權流程介紹下來，不同流程分別有不同的適用情境與授權要求，也會有需要不同的參數。我們可以發現不管再進行哪種要求，一定都需要有 `Response_type` 參數，來說明使採用哪種流程，第一種目前大眾最常使用的授權代碼授予流程需要進行兩種要求，分別為授權要求及存取令牌要求，在進行要求時所需要的參數最複雜，標準也推薦客戶端使用 `state` 參數，用來證明目前客戶端自身狀態。

除了存取令牌外，規範內也提供刷新令牌(Refresh Token)，當存取令牌期限過後，可使用刷新令牌換取新的存取令牌，主要用途是防止存取令牌外洩被盜用的安全問題。除了存取令牌外洩的議題外，規範內還提出了十六點的安全性問題，本研究將其簡單整理，可發現 OAuth2.0 規範主要是針對使用者授權第三方擁有其受保護資料的流程，但針對雙方身分驗證方面的規範並未詳細定義，其中規範內所列出的安全性問題有七項都可能因為無身分驗證導致，當攻擊者如果盜取存取令牌並偽裝成客戶端要求授權資料，因伺服器只認存取令牌，會導致用戶資料外洩。本研究針對此議題進行探討，並應用零信賴架構來進行改善。

2.2 OAuth Single Sign On

單一登入機制(Single Sign On , SSO)提供使用者在瀏覽第三方應用程序時可以無縫進行身分驗證，使用者只要登入有提供單一登入機制的應用程序其中之一，就不需要在另一個應用程序進行登入，不論這些應用程序提供什麼服務或平台，使用者皆可自動登入，不僅讓使用者可以輕鬆使用眾多服務，也可避免使用者還需要再額外記住多組帳號密碼。我們可依據圖 6 來看單一登入機制的概念[8]，主要是有一個所有應用程序都信賴的中央服務器，當第一次登入時，各應用程序都會轉址到中央服務器進行登入，並且創建一個 cookie，當我們第二次登入時，一樣會藉由轉址到中央服務器進行判斷，如果裡面已有 cookie 在，就代表我們曾經登入過，就會直接導向應用程序。

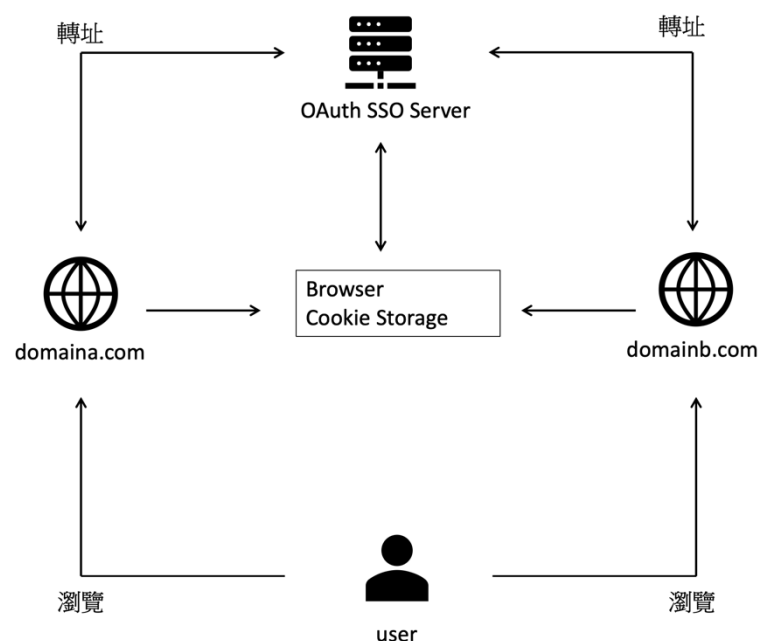


圖 6 OAuth SSO 概念

(資料來源：整理自 Auth0(2016)。How to Implement Single Sign On)

OAuth SSO 主要是在單一登入機制上多了授權的概念，當第三方應用程式提供某應用程式的單一登入機制時，使用者可授權第三方應用程式去存取此應用程式內的使用者資料，例如使用者想在某購物網站上購買物品，而購物網站為了減輕使用者要再多記一組帳號密碼的負擔，而提供了 OAuth SSO 服務，使用者

可登入 Google 的帳戶來授權此購物網站使用自身的資料，這樣一來，購物網站不需要額外開發會員系統或保存會員資料，而只要透過使用者授權就可依據使用者的 Google 帳戶來認證使用者身分。圖 7 呈現的為 OAuth SSO 流程[9]，整體來看與 OAuth 授權流程無太大差別，主要是在拿取存取令牌後，就可透過此存取令牌到其餘應用程序進行資料要求或使用該應用程式。

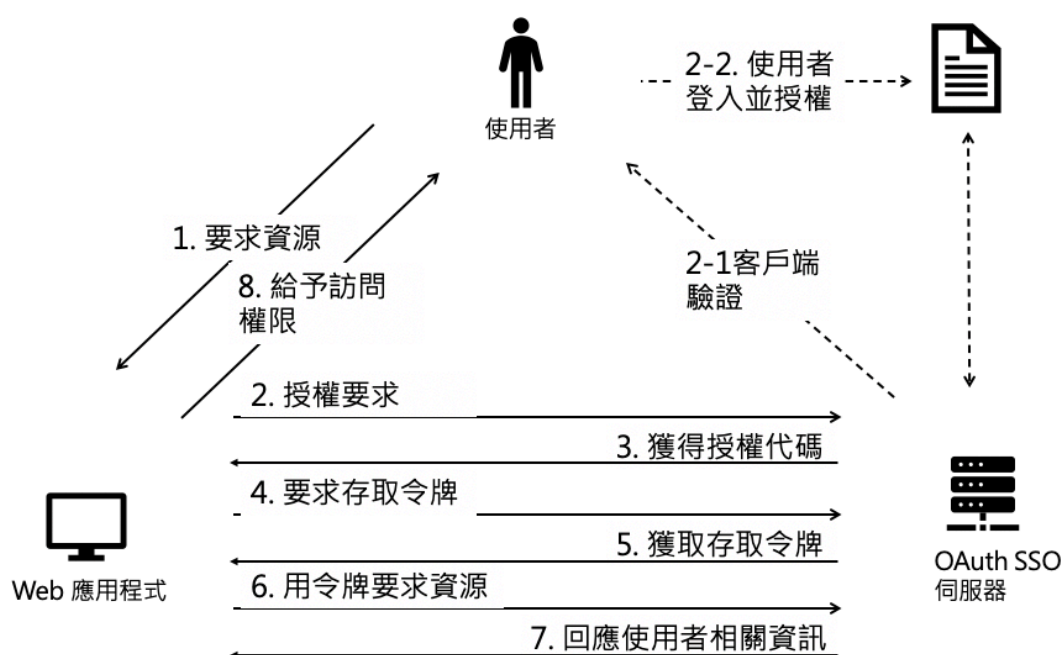


圖 7 OAuth SSO 流程

(資料來源：miniorange(2021)。OAuth2.0 Server/OpenID Connect Server)

OAuth SSO 雖提供了使用者很高的方便性，但在整體使用上或傳輸流程上還是有些為地方需要注意(Hossain et al.,提出)，像是傳輸過程中是否有使用 HTTPS、授權流程中是否有使用未定義參數、授權中使用到參數的使用及存取是否皆有保護、能否預防 CSRF 攻擊等[10]，在 OAuth SSO 的流程中，我們需要注意傳輸安全、參數使用、攻擊者偷竊或假冒等攻擊，以及是否有結束機制，假使攻擊者在授權流程中竊取了授權代碼，並在使用者使用前使用，便可冒充使用者換取存取令牌換取敏感資料，又或者使用者在登入無法透過關閉頁面來結束會話，有可能會因為無結束機制，導致攻擊者有利可圖。OAuth SSO 與原本 OAuth 授權流程差異不大，但如果攻擊者有心利用流程中的漏洞，會因此照成更大的資安問題，因此在使用 OAuth SSO 時需要更加謹慎。

2.3 Open ID Connect

OpenID Connect(OIDC)與 OAuth2 類似，OAuth2 主要是定義授權框架，而 OIDC 則是在授權框架上多了身分驗證的流程，OIDC 於 2014 年發布[11]，內容主要包括 OAuth2 及 JAON Web Token(JWT)。原本 OAuth 僅定義了授權流程，對於身分認證則無多加著墨，但以授權流程來看，如果有攻擊者偽冒身分，會有資料外流的資安風險，因此 OIDC 則在既有流程上允許授權伺服器驗證使用者身分，避免使用者身分被盜用而同意授權將資料給不信賴的第三方應用程式。

因為是基於 OAuth2 授權流程上增加了身分驗證，OIDC 流程中有許多角色跟 OAuth2 差不多，指示因應功能不同，名字有變動，分別為：OpenID Provider(OP)、Relaying Party(RP)、End User。OpenID Provider 類似 OAuth2 的授權伺服器，但有加上驗證使用者身分功能，Relaying Party 指的是 OAuth2 的客戶端，來請求終端使用者的身分驗證，以及從 OpenID Provider 來獲得服務，End User 則是終端使用者，使用 Relaying Party，並且能通過 OpenID Provider 的身分驗證，我們可透過圖 8 來了解 ODIC 的流程。

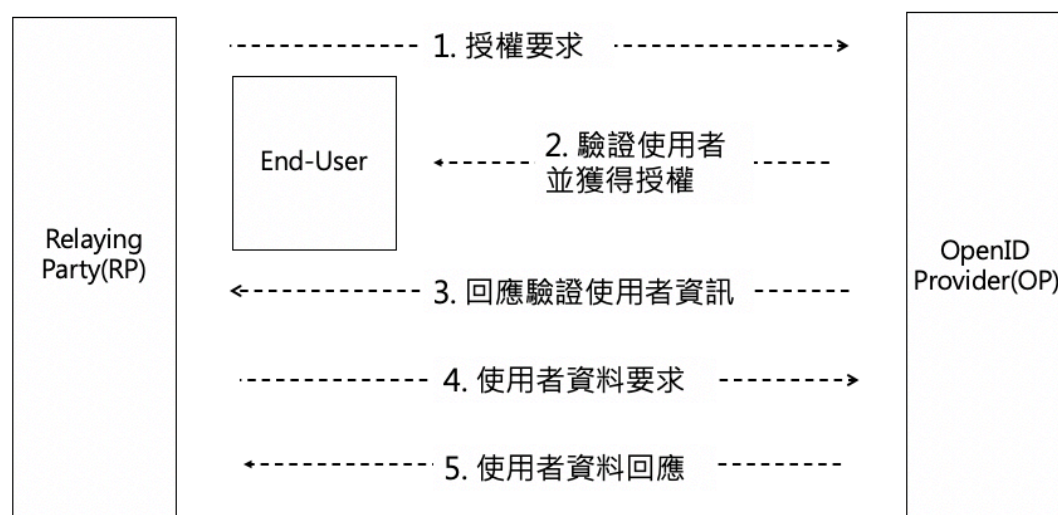


圖 8 OpenID Connect 流程

(資料來源：Sakimura, Bradley, Jones, Jay. (2014). OpenID Connect Discovery)

主要流程分為五個步驟，(1)RP 送出要求給 OP，之後(2)OP 驗證終端使用者身分並獲得使用者授權，(3)OP 將存取令牌及 ID Token 回應給 RP，(4)RP 可

透過存取令牌要求使用者資料，(5)OP 回應使用者資料。流程上跟 OAuth2 差不多，都是需要先客戶端提出要求，企業伺服器驗證並獲得使用者授權，給予客戶端令牌來存取使用者資料，但唯一的差別在於 OIDC 在驗證使用者身分後，會額外傳一個 ID Token 給客戶端，透過此身分令牌讓客戶端能夠驗證終端使用者身分。此 ID Token 是一個安全令牌，並透過 JSON Web Token(JWT)表示，內部參數包含：發布者標示(iss)、主體標識符(sub)、令牌的受眾(aud)、令牌期限(exp)、令牌發行時間(iat)、用戶身分驗證發生時間(auth_time)、隨機數(nonce)、身分驗證上下文參考(acr)、身分驗證方法參考(amr)及授權方(azp)，透過這些參數聲明，此 ID Token 可提供身分驗證的完整性、不可否認性。

OIDC 雖然是建立在 OAuth2 之上的身分認證及授權協定，但主要多了 ID Token 是在驗證終端使用者的身分，讓第三方應用程式在使用時能夠確保使用者的身分是否為使用者宣稱的身分，可是在授權協定中，除了企業端的授權伺服器我們可信賴外，終端使用者及提出要求的客戶端我們都無法確認其是否有被攻擊者入侵或仿冒，因此 OIDC 協定僅提供了輕量級的身分驗證，也就是關於終端使用者的驗證，卻無針對另外一方，也就是第三方應用程式端進行身分驗證，即使我們可以驗證終端使用者身分，但第三方應用程式如果被偽造了，使用者的資料仍可能會被外洩，因此我們還是需要針對授權流程中身分驗證不足的部分進行改進。

2.4 零信賴架構

因應目前企業基礎架構越來越複雜，企業除本地基礎架構外還會有雲端服務、遠距工作及移動式個人設備(Bring-your-own-device，BYOD)等，這導致傳統使用基於邊界的網路安全性方法已不夠防止攻擊者攻擊，一但攻擊者突破邊界，就無法阻擋他進一步竊取企業內部機敏資料。因此，Zero Trust Architecture(ZTA)零信賴架構的網路安全模型被開發出來，能防止資料外洩並限制攻擊者突破邊界後的橫向移動，保護企業所有資產的數據及使用者端的訊息。

零信賴架構由 NIST 於 2020 年八月提出的 NIST 800-207[6]，「零信賴」一詞起源於傑理科論壇(Jericho Forum)與去周邊化(deperimeterization)概念，當反邊界化的思想及限制網路位置的隱式信任概念不斷發展和完善，這概念就形成了。而較為具體的概念則是在 2010 年由 John Kindervag 在 Forrester 提出的論文[10]，論文內主要論點為：

1. 不再有信賴與不信賴的邊界在我們的安全裝置上
2. 不再有信賴與不信賴的網路
3. 不再有信賴與不信賴的使用者



零信賴(ZT)旨在面對受到威脅的網路時，將系統和服務的訪問權降到最小特權，保障不確定性降到最低；而零信賴架構(ZTA)則是企業的網路安全計畫，利用零信賴的概念將組件、工作流程、訪問策略都包括起來。企業可將零信賴架構應用於企業資源與資料安全的端到端方法，如身分、憑據、訪問管理...等資源，會專注保護資源，永遠不會有隱式信任，只要有訪問者，一律需對其進行評估，確保訪問者是安全的，才可以拿取資源。假定企業環境中存在著攻擊者，將企業內部及外部的環境視為一樣，沒有任何可信賴的環境，對於要訪問每個企業資產的使用者，不論是否為企業員工或外部訪客，都需認證其身分和確認安全狀態才可授權其訪問，透過不斷分析和評估資產及業務功能所面臨的風險，制訂保護措施減輕這些風險，同時在零信賴的狀態，會將資源的訪問最小化，而能訪問的使用者也會不斷的被認證身分和狀態。

零信賴原則有七個[6]：

1. 所有資料來源與運算服務都被視為資源
2. 不論網路位置在哪，所有的通訊都要受到保護
3. 對於個別企業資源的存取要求，應該要以每次連線為基礎去進行許可
4. 資源存取由動態策略決定(基於客戶端識別、應用服務、及要求請求資產的狀態或行為或環境屬性動態決定)
5. 企業監控與衡量所有擁有和相關資訊資產的正確性與安全狀態
6. 所有資源的身分驗證和授權都是動態的，要在允許訪問前嚴格執行
7. 企業應該要儘可能收集有關資訊資產、網路架構、骨幹，與通訊的現況。並用這些資訊來增進安全狀態

為了達成零信賴原則，減少對訪問者的不確定性，重點在於「認證」、「授權」、「縮小信任區」，同時要最大程度的減少認證機制中的時間延遲來保持可用性，企業可將資源放在隱式信賴區域中，當有人要訪問企業資源時，要通過策略決策點(Policy Decision Point ,PDP)及策略執行點(Policy Enforcement Point ,PEP)授予訪問權限。

策略決策點及策略執行點會透過判斷來讓訪問者可訪問資源，意味著零信賴架構需要進行身分驗證和授權，在訪問者請求時，要如何判斷訪問者的安全性？如何給定訪問者身分的信賴程度？是否允許訪問者訪問資源？用來請求的設備是否具有適當的安全狀態？是否有其他因素需考慮且是否會改變對訪問者的信賴程度？以零信賴架構來看，策略決策點可在區分為策略引擎及策略管理，策略引擎可決定是否給予訪問者訪問權，並與策略管理者合作，策略引擎做決策，策略管理者執行該決策；策略管理者除了依據策略引擎所做的決策來建立或關閉主體與資源間的溝通路徑外，也會生成客戶端訪問企業資源時所用的身分驗證令牌或憑證；當前述兩者做好決策並執行後，策略執行點會負責啟用、監視及最終結束主體與企業間的連接，只要通過政策執行點，後面就是企業資源的信任區。

總體來看，企業需要針對資源訪問來制定基於風險的動態訪問策略，同時建立一個系統確保這些訪問請求的正確性，也就是說，企業不應該依賴隱含的信任度。以機場為例，通過安全檢查的乘客可在機場內部遊走，這些乘客都被視為是可信任的，以 ZTA 角度來看，乘客就是要求訪問資源的訪問者，安全檢查就是

PDP/PEP 的角色，當乘客通過安檢後，進入到的機場內部就是企業的資源，也就是隱式信任區，在機場內部的乘客都被視為是可信任的，因此隱式信任區的範圍應盡可能的越小越好。

零信賴架構的核心元件如圖 9，大體分為控制平面及資料平面，控制平面負責邏輯組件間的通信，資料平面負責應用程式資料的通訊。政策決策點被視為邏輯組件，主要由兩個元件組成，政策引擎(Policy Engine, PE)及政策管理者(Policy Administrator, PA)；資料平面上會有主體、系統、政策執行點及企業的資源，其餘組件則是可提供一些資料作為政策引擎在制定訪問決策時可使用的輸入，如持續診斷和緩解系統(Continuous diagnostics and mitigation system, CDM)收集企業資產當前訊息或威脅情報源(Threat intelligence feed)會有內部及外部來源的訊息、數據訪問策略(Data access policies)會提供有關訪問企業資源的屬性、規則及策略。

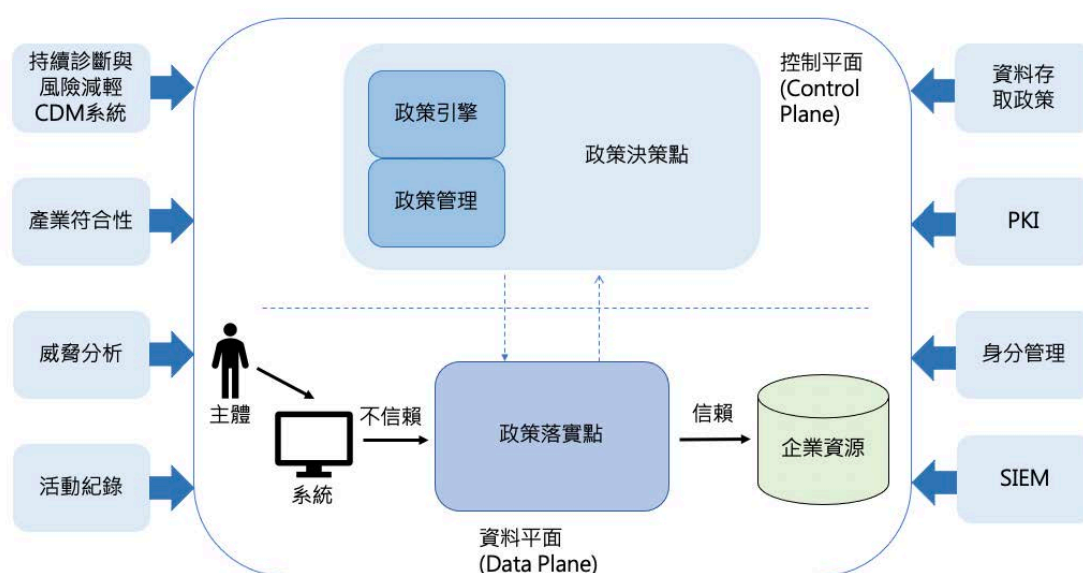


圖 9 ZTA 核心元件圖

(資料來源：整理自 Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2019). Zero trust architecture (No. NIST Special Publication (SP) 800-207 (Draft))

在參考 ZTA 幾大原則及 ZTA 核心元件圖後，企業如要部署 ZTA 時，需要參考自身不同的網路設置模式來決定要如何部署，像 Google BeyondCorp[12] 是 Google 設計的零信賴模型，主要要讓員工不需要透過虛擬私人網路(Virtual private network, VPN)也可以在不被受信賴的網路中工作，PagerDuty[13]設計的零信賴網路主要處理公有雲環境中，多個供應商的安全互動。

在基於 NIST800-207 中，將 ZTA 部署模型分為四種，基於設備代理/閘道器的部署(Device Agent/Gateway-Based Deployment)、基於限制區域的部署(Enclave-Based Deployment)、基於資源門戶的部署(Resource Portal-Based Deployment)、設備應用沙箱(Device Application Sandboxing)。除此之外，Jason Garbis 及 Jerry W.Chapman 發表的 Zero Trust Security：An Enterprise Guide[14]中有另外提及兩種不同的部署模式，雲端部署(Cloud-Routed Deployment Model)及微分割部署(Micro segmentation Deployment Model)，不同部署方式各有不同的優缺點，而考量到其中幾種部署模式與本研究所探討之 OAuth 授權框架較無相關，僅介紹幾種部署模式。

表 2 不同部署模式的比較

	設備代理/閘道器的部署	限制區域的部署	資源門戶部署	雲端部署
介紹	企業上的「每個」資源前都配置閘道器，客戶端需安裝代理才能存取企業資源	企業上針對「不同資源區域」配置閘道器，客戶端需安裝代理才能存取企業資源	客戶端不需安裝代理，可藉由企業的代理門戶負責客戶端要求	客戶端透過雲端環境上的政策決策點判斷是否可存取
優點	- 應用程式和網路流量的端對端控制 - 閘道器背後有範圍極小的隱式信任區	•PEP 能簡單部署 •減少 PEP 的部署 •能較好處理臨時的工作負載和動態環境 •PEP 可在 DMZ 上運行，並充當自然入口點	•較為靈活 •設備不需要安裝設備代理	•簡單(不需更改防火牆) •可減少企業的營運開銷 •僅適合遠程用戶
缺點	•需要在使用者設備及資源上部署 PEP •資源元件與 PEP 可能會發生技術衝突 •PEP 需能被部署在各種過時或舊型的系統上 •每個資源都需有專屬的閘道器 •PEP 必須可見並且能夠讓遠端用戶可用	•淺在大量隱式信賴區 •PEP 代表進入企業網路的新型入口點	•無法確保設備安全性，僅能在設備連接到 PEP 門戶時對其掃描 •企業沒有設備資訊的完全可見性或對資產的任意控制權	•可以在無適當的安全性下部署 PEP •增加用戶流量的延遲不適合訪問本地資源的本地用戶 •會有較大的淺在信任區

(資料來源：本研究整理)

圖 10 為基於設備代理/閘道器的部署，在此模型中，政策執行點被分為兩個部分，一個位於資源前面，一個留在資源上，例如在每個企業發行的資產上都安裝一個代理來負責協調連接，而每個資源都有一個閘道器，充當代理的角色，而這兩個代理也可視為政策執行點。因為資源僅會與此閘道器通訊，因此在閘道器尚未確保請求方是安全時，不會讓此連線通過，企業系統上的代理與資源上的閘道器會與政策管理員互相通信，當有人提出請求時，會請在控制平面上的政策決策點之中的政策引擎評估是否安全，才會同意是否該讓此人連線。舉例來說，當有使用者使用企業發行的筆電嘗試要連接到企業的內部資源，此訪問請求會先有本地代理接收，將請求轉發給政策管理員，政策管理員會將其轉發給政策引擎進評估，政策引擎會基於像是 IP 位址、端口訊息、會話密鑰等安全訊息來決定是否同意，當請求被同意後，政策管理員就會與本地代理與相關資源的閘道器進行通信，並讓此訪問者訪問此企業資源。此模型適合有設備管理程序即可用閘道器通訊的離散資源企業，如有企業不想實施移動式個人設備(BYOD)，也適用此模型。

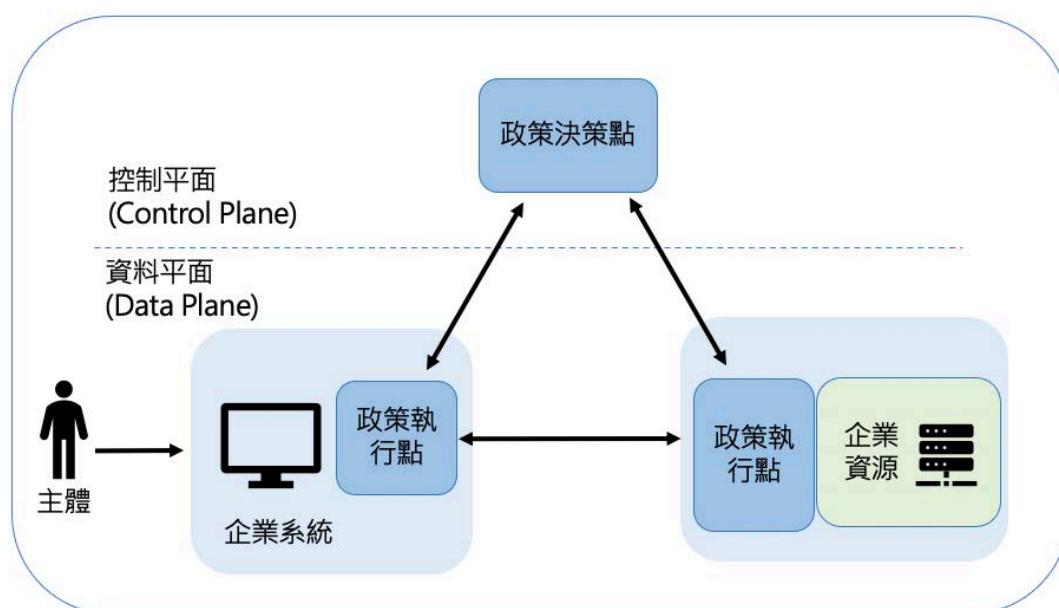


圖 10 基於設備代理部署模型

(資料來源：整理自 Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2019). Zero trust architecture (No. NIST Special Publication (SP) 800-207 (Draft)))

圖 11 為限制區域模型，為前述模型的變體，主要是考量閘道器組件無法針對單個資源作配置，因此將閘道器配置在資源區域的邊界，整體模型與基於設備

代理的部署模型一樣，差別只在企業將使用雲服務的服務或資源放置在私有雲上，而私有雲部署於閘道器後面，此模型對於有遺留系統或針對數據中心無法配置單個閘道器的企業很有用，但缺點在於閘道器無法單獨保護每個資源，而是保護資源集合，要求請求的主體也可能看到他們無訪問權限的資源。

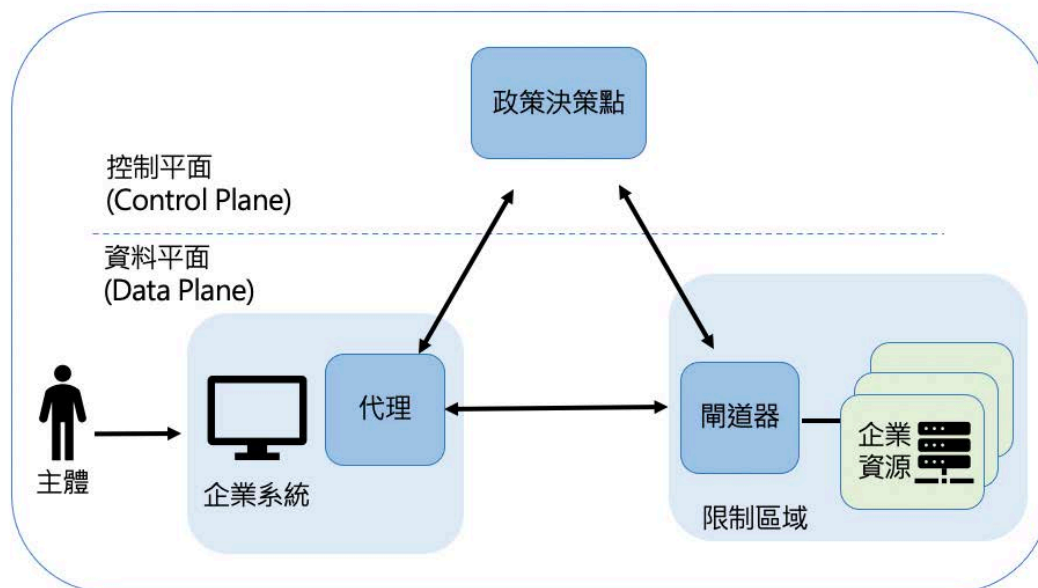


圖 11 基於限制區域部署模型

(資料來源：整理自 Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2019). Zero trust architecture (No. NIST Special Publication (SP) 800-207 (Draft)))

圖 12 為資源門戶模型的部署模型，在此模型中，政策執行點主要是充當主體請求閘道器的單個組件，閘道器可以用於單個單元也可以用於某功能資源集合的安全區域，例如主體要求進入私有應用程序的私有雲或資料中心的閘道器。此模型與其他模型差別在於，不需要在所有客戶端設備上都安裝軟體組件，對於實施移動式個人設備(BYOD)的企業來講，此模型應用會比較靈活，企業管理員不用在使用前確保每個設備是否都具有適當的設備代理，而改由設備請求時來推斷出訊息。

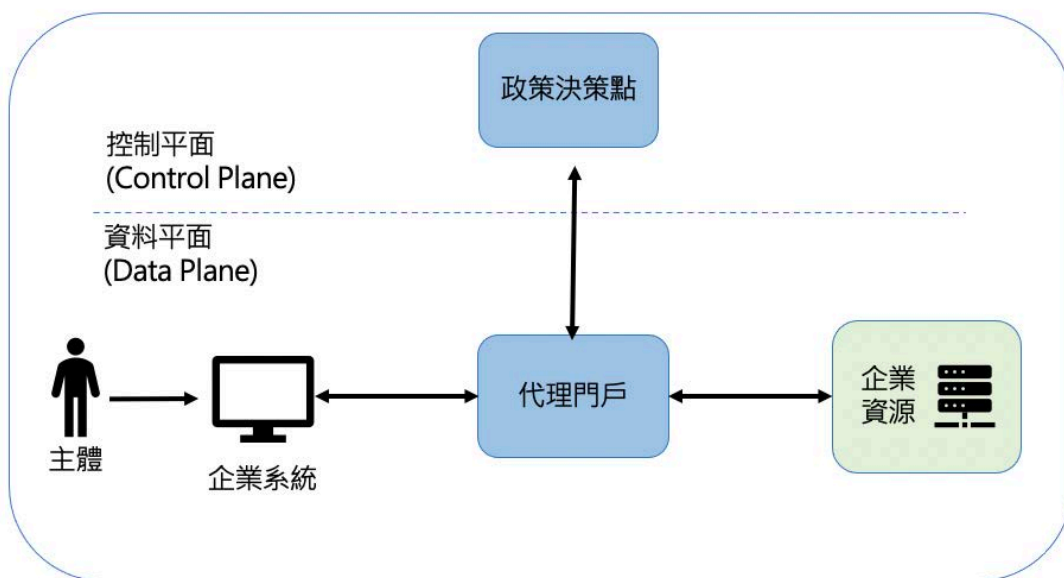


圖 12 資源門戶模型

(資料來源：整理自 Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2019). Zero trust architecture (No. NIST Special Publication (SP) 800-207 (Draft)))

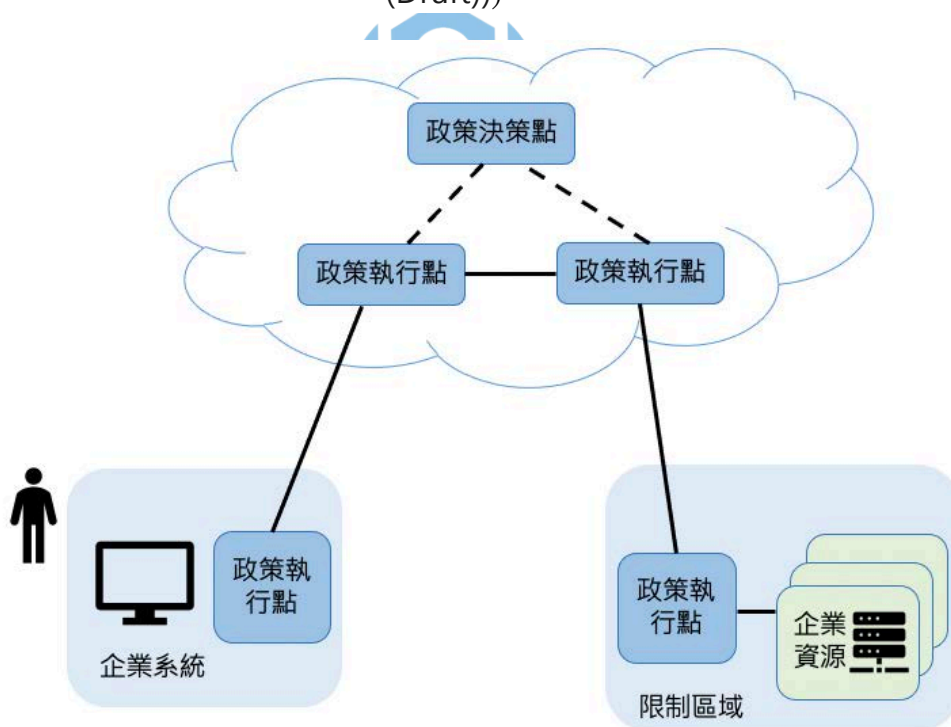


圖 13 雲端部署模型

(資料來源：整理自 Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2019). Zero trust architecture (No. NIST Special Publication (SP) 800-207 (Draft)))

圖 13 為雲端部署模型，主要為 Zero Trust Security: An Enterprise Guide[14] 書中提出的，在此模型中，主體提出的要求需要先經過雲端環境，並通過政策決策點的身分驗證，通過後會將要求傳送到地理位置最近的雲端政策執行點，此時雲端的政策執行點會在連接企業資源前的政策執行點。這種模型是一種常見的方法，許多商業供應商都會將其作為服務來運行。

上述幾種模型為企業要部署零信賴架構時可參考的其中幾類模型，企業可考量自身網路部署環境及用途來選擇較適合的部署模型，而對於這些部署的企業，要將政策引擎視為大腦，將政策執行點的信任演算法是為思考過程，信任演算法是讓政策引擎用來做評斷是否同意授權或拒絕資源訪問的過程，可參考多個輸入來源如：訪問者的要求、訪問者是誰、訪問設備的狀態...等許多參考因素來做為評斷，假使某訪問者的設備無裝防火牆或未安裝最新的補丁，信任演算法就會降低，讓政策引擎給予此使用者較低的存取權限；如某訪問者有安裝防火牆且又通過設備驗證，政策引擎就可給於其最高的訪問權限。

目前有許多企業也開始使用 ZTA 來部署自己的公司，如 Google 及 PagerDuty，我們可藉由兩家公司的部署模式的不同來了解不同企業環境該如何部署。



1. Google BeyondCorp

Google 於 2010 年初期考量到傳統的邊界安全模型防禦力越來越不符合需求，當企業發展為全國企業，有好幾百萬的員工需要在遠端工作，公司內部整天也有許多人員來訪，以往的邊界安全模型無法防禦好系統安全，因此 Google 開始設想如何保護好企業的安全狀態，確保員工不論在哪都可使用企業資源，同時在這樣的情況下，也要保護好客戶提供給 Google 的機敏資料安全性。因此 Google 開始構想零信賴架構，期望能夠將存取權限控管從往路邊界轉移至個別使用者，BeyondCorp 在 2014 年被 Google 提出，主要有三大原則[12]：

1. 服務存取權限不在受限於使用者連線的網路
2. 根據使用者及裝置的相關資訊來授與服務存取權
3. 不論要使用哪種服務，都需經過驗證、授權及加密程序

這三大原則讓員工不在受限於特權內網的概念，所有員工可以透過自身

使用者的身分憑據及設備驗證來決定存取公司資料的權限，使遠端工作者能利用的企業資源與本地工作者一樣。

圖 14 為 BeyondCorp 的主要組件，確保使用者及設備只有在經過身分驗證後才有權存取企業應用程式，在這些組件當中，主要會利用「設備資產清單資料庫」，來安全識別並追蹤所有受管理的設備，確保只有由企業採購且被企業管控的設備才能存取內部應用程式，同時在整個設備生命週期中，公司會定期追蹤設備的變更，分析目前設備的安全性為何。

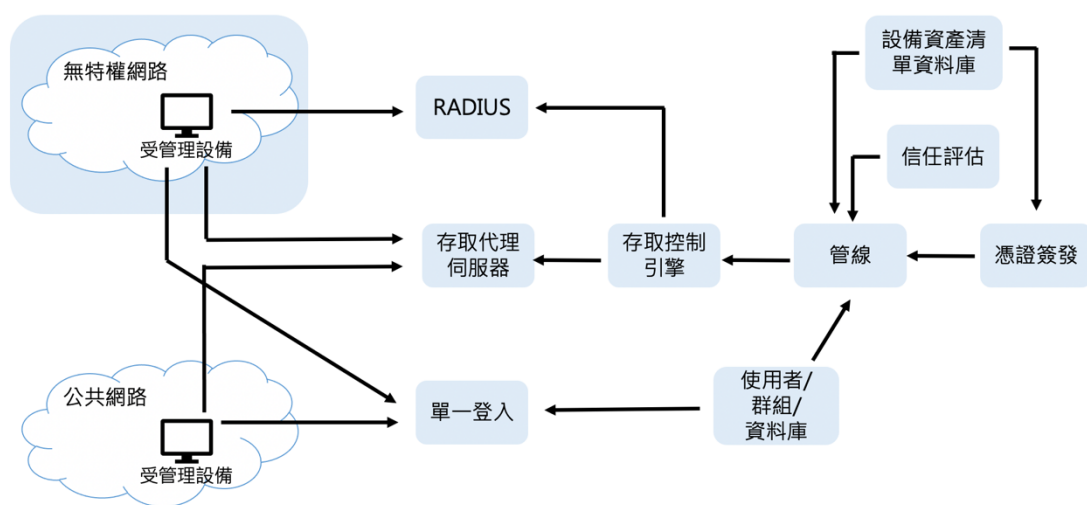


圖 14 Google BeyondCorp 組件

(資料來源：整理自 Ward, R., & Beyer, B. (2014). BeyondCorp: A new approach to enterprise security.)

除了設備身分驗證外，Google 也會透過靜態規則和啟發式方法來動態推斷要授權給使用者或設備的信任度，例如設備是否有安裝最新的作業系統修補程式或依據使用者存取資料的位置來給予不同信任。Google 會在每個應用程式上配置存取代理伺服器，由代理伺服器裡的存取控制引擎來根據不同請求及相關資訊來決定企業應用程式可給予多少層級的授權要求。

舉例來說，我們可透過圖 14 來了解有使用者要訪問 Google 應用程式時，會經過的流程。使用者一開始提出要求時需附上設備憑證，此要求首先會到代理伺服器，當代理伺服器發現無法識別使用者時，會將使用者導向登入系統，讓登入系統驗證使用者身分，之後發出存取令牌，並導回代理伺服

器。這時代理伺服器擁有設備憑證能驗證設備正確性，及令牌驗證使用者身分，就可讓存取控制引擎驗證使用者屬於哪個群組、設備是否為良好的受管理設備、使用者及設備的信任級別為何，當檢查通過後，就可將此要求傳遞到後端進行服務。當提出 BeyondCorp 後，Google 要開始將企業內的組件遷移至 BeyondCorp，但要一次將所有資源遷移過去會有相當大的風險，因此 Google 投入大量資源，分批進行遷移，到如今幾乎所有機構都可使用 Google 提供的零信賴架構模型。

Google BeyondCorp 目前已經到收尾階段，我們可藉由此案例來了解一間企業要由傳統邊界模型改變成零信賴架構需要花費大量時間跟精力，在轉移過程中，需要所有人的積極配合才能讓企業順利完成，而藉由 Google 的案例，其餘企業可參考他們的作法設想自身該如何實作零信賴架構，在思考過程中，權衡零信賴的動機與設計原則。

2. PagerDuty

PagerDuty 是一家為資訊部門提供事件應變能力的雲端平台，使用者可事先設定監控系統，當緊急狀況發生時，PagerDuty 就會能夠及時通知，讓相關人員可更快速的解決問題，例如系統發現在平台上的影片突然出現問題，PagerDuty 就會即時通知該企業相關技術部門，讓他們緊急處理。考量到 PagerDuty 的運營模式，公司對於客戶的資料隱私有相當高的要求，因此他們於 2013 年開始嘗試建立零信賴模型，希望能處理眾多供應商間公有雲環境的伺服器的安全互動。

PagerDuty 建構零信賴架構的組件與 Google 較不一樣，PagerDuty 主要是透過組態管理工具 Chef 來管控系統中所有的虛擬機，透過組態管理工具可以快速導入零信賴模型，除了可將存取管理政策集中管理外也可隨著部署的數量增加來擴展；但是透過組態管理工具也會有缺點，需要確保所有節點都照著政策來執行，因此需要持續監控政策的有效狀態。

為了確保每部主機都可以保護到，PagerDuty 讓 Chef 依據對系統的了解產生 Iptables 組態，每台主機都要建置自己的 Iptables，針對不同角色的 IP 位置給予不同權限，如果該流量不符合規則的白名單，就丟棄。而針對零信賴架構下每次連線都需要驗證身分這點，PagerDuty 則透過 IPsec 實作主機對主機的網狀網路，讓所有封包都是由系統中的每個節點加密和驗證身分，這樣的配置方式其實與 ZTA 中的微服務分段部署模型相似。

除了上述所提到的 Google BeyondCorp 及 PagerDuty 外，市面上目前以有一些廠商針對第三方授權結合零信賴提出一些想法，有些主要針對網路流量來進行監控及分析、有些則是會提出雙因子認證(two factor authentication, 2FA)來確保使用者身分，如思科(Cisco)所提出的 Duo 2FA[15]。雙因子認證的兩個因素會參考使用者密碼、使用者指紋或一次性密碼等，如果以使用者所擁有的東西判斷，如密碼或 OTP，也僅限在獲取令牌時能確保使用者身分，當令牌被偷且被拿來利用時，也無法確認其身分。因此我們需要在使用令牌時，也就是每次要求時，都要確保來的人是使用者，如果將令牌作為雙因子認證的其中一個驗證因素，當攻擊者已成功竊取令牌，也代表攻擊者可能已經能拿到使用者的其他資料，如密碼等，因此需要選擇另外一個通道來作為另外一個驗證因素，思科提出的 Duo 2FA 分別就有提出兩種方式來作為第二個驗證因素，Duo Push 跟 U2F，Duo Push 是讓使用者以行動裝置做一個點擊的動作來做確認；U2F 則是另外使用 USB。雖然 Duo Push 很方便可以快速驗證身分達到認證效果，但因為是使用移動設備來驗證，無法得知使用者使用的設備情況，導致不能動態調整風險，而 U2F 使用 USB 來做驗證，如若今天 USB 遺失，就無法使用，變動就大。

藉由前面所介紹的 OAuth、OAuth SSO、OIDC 及零信賴架構等，我們可以了解傳統基於令牌為驗證的方式會有一定的資安風險，以往透過信賴令牌的方式僅限於在企業內部進行存取，在遠距辦公方式興起的今天，需要重視傳統只重視邊界的問題，新環境需要以零信賴為主，重新設計身分認證，因此我們可針對零信賴架構所提出的七大原則暨部署模式來改善及避免傳統令牌驗證鑑別方式的資安問題，針對 token-based 方法進行強化，像是我們需假設整個企業的私有網路皆不能視為隱式信任區、當有遠端連線時，企業要假設遠端連線網路與本地網路是敵對的、如要請求企業資源時，也須通過政策執行點對每個資產的安全狀態評估。

第三章 系統需求分析與定義

3.1 問題定義

現今存取控制多採用 OAuth，讓使用者可在不洩漏自身帳密就可授權第三方使用，以 OAuth 授權框架來看，我們可發現 OAuth 本身是以「存取令牌」為基準來進行授權，只要授權伺服器或資源伺服器驗證存取令牌的正確性，就會將受保護資料提供給存取令牌提供者。當企業採用 OAuth SSO 讓員工可使用單一登入進入企業所提供服務後，所有系統都會依據存取令牌來驗證員工身分，並讓員工使用多項企業服務或提供其要求的受保護資料，如圖 15，企業應用服務透過存取令牌獲取使用者個人資料進行身分識別，而後使用者就可存取應用服務。

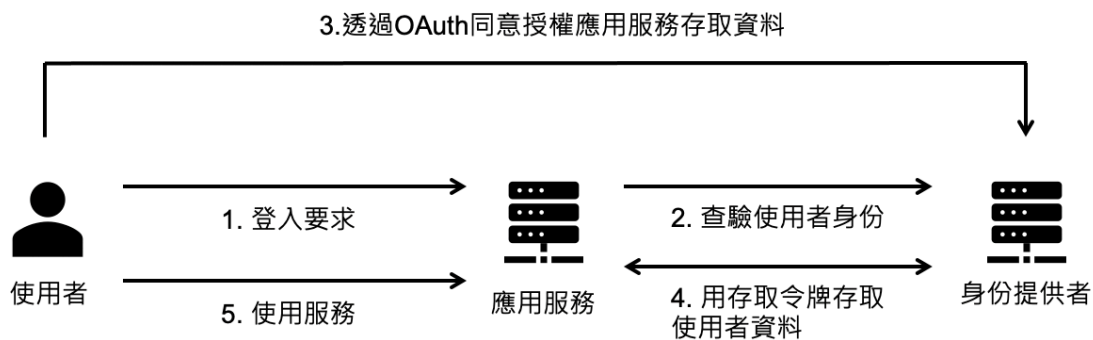


圖 15 OAuth SSO 情境

(資料來源：本研究整理)

但現今辦公方式改變，各種新興存取方式的發展下，傳統企業使用令牌來信賴使用者的方式不在適用，要針對以往信賴的邊界的做法作出改善，提出強化方法，否則攻擊者可透過遠端竊取使用者存取令牌並假冒使用者存取資料，如圖 16，攻擊者可利用此令牌暢行無阻，使用企業內的各服務或於各服務間取得敏感資料。透過上述情境，讓我們了解傳統以 Token-based 作為鑑別方式的做法會有令牌被竊取造成資料外洩的資安風險，雖然 2014 年有提出 OpenID Connect(OIDC) 協定[11]，主要在 OAuth 的基礎上，另外定義了身分驗證流程，在 OIDC 的協定中，多了一個 ID 令牌，用來識別使用者身分，讓第三方可信賴使用者；而對於第三方的身分驗證並沒有過多的著墨，因此依然無法解決 OAuth 授權框架上客戶端驗證的問題，也無法對傳統邊界作法進行強化，而零信賴架構是針對傳統

強化邊界的方法，透過持續驗證及不信賴任何人來確保所有連線安全，整體基本是將所有連線視為攻擊，當確保連線端的安全性後才能准許連線，與 OAuth 僅透過令牌做身分鑑別不同，因此本研究針對 OAuth 傳統以令牌為基礎的鑑別方式不支援零信賴架構原則的問題進行探討。

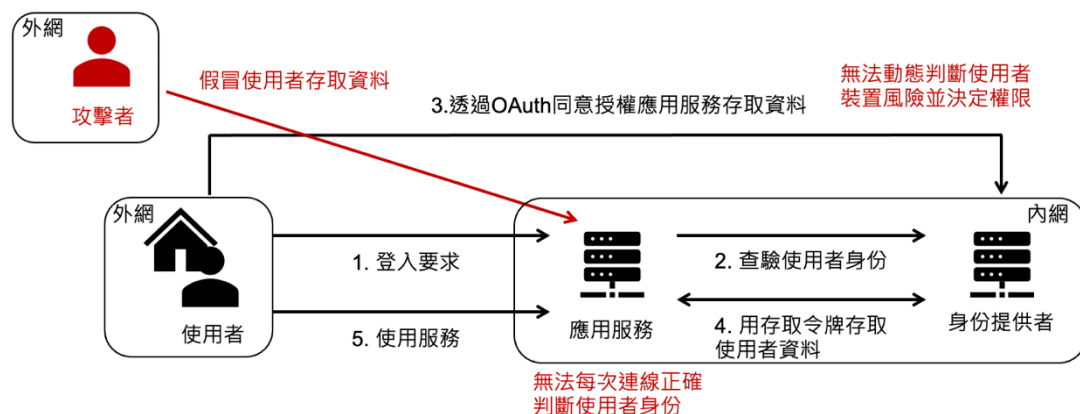


圖 16 OAuth SSO 攻擊者情境
(資料來源：本研究整理)

當我們將零信賴架構框架套進 OAuth 看，會發現有幾個問題存在：

- 無針對每筆連線進行驗證並識別
- 客戶端驗證不足
- OAuth 無客戶端內容資訊來判斷不同的風險等級

從上述所提出的想法及第三方授權套入零信賴架構會產生的問題來看，我們可以提出一個改善的框架，讓攻擊者即使跨邊界進行存取也無法得手，整個框架既能強化 token-based 方法，讓遠距工作等存取方式能夠繼續發展，也能夠讓 OAuth 支援零信賴架構原則，對於授權伺服器來說，我們需要定義發存取令牌的規範，要讓它針對每一筆連線或存取要求進行識別，確保是正確的客戶端要求存取資源；同時在客戶端要求資料時，需要提出一些自身的內容資訊，讓資源伺服器能夠針對客戶端的設備資訊來判斷出要給予哪個風險等級的資料。

3.2 需求分析

以 OAuth SSO 來看，當一個未知使用者透過瀏覽器或使用者代理跟 OAuth SSO 伺服器提出要求時，伺服器會先請使用者登入，成功後，伺服器會傳送授權代碼給瀏覽器，而瀏覽器再透過此代碼換取授權令牌，並透過授權令牌要求使用者資源，最後，在使用者成功登入且驗證通過後，就可獲得授權權限。整個 OAuth SSO 流程過程[9]中，需要注意傳輸過程的安全性[10]、以及授權代碼和授權令牌等參數的存取與使用也要特別注意，另外，當使用者能夠使用 SSO 登入並存取相關資源時，我們也要注意此會話的結束機制，避免使用者被冒充時，依然能夠存取敏感資料。

在了解 OAuth SSO 等傳統以令牌為基礎的鑑別方式後，為了要提高其安全性，並讓目前新興存取方式能夠發展，我們可以參考零信賴架構的幾項原則並提出改善方法，讓傳統以令牌為基礎的鑑別方式能夠支援零信賴架構原則，同時解決企業過往普遍信賴內部使用者到現今因遠距存取發展導致使用者已無內外之分的问题。因辦公模式的改變，以往使用者都會在企業內部進行存取要求，企業僅需要透過令牌就可直接驗證，而現今使用者可能會從自身家中或各地連回企業，並提出存取要求，只要有攻擊者偽造成使用者突破企業邊界防護，就可在企業內部暢行無阻，因此為了要強化企業邊界及對提出存取要求的使用者驗證，本研究參考零信賴架構原則來提出需求。

在考量 OAuth SSO 流程及零信賴原則下，本研究整理出需求如下：

- R1：需要針對每次登入週期的資料存取進行要求許可
- R2：要確保送出要求的客戶端為收到存取令牌的客戶端
- R3：能夠確保客戶端設備的安全性並依據設備安全性提供不同等級的受保護資料
- R4：能收集客戶端設備狀況來增進安全狀況

本研究主要是以零信賴架構原則來看，透過持續驗證及不信賴任何人來確保信賴區內資料的安全，企業需要對每次連線進行「授權」，確認要求連線者的身分才可授權他進入企業信賴區存取資料，資料存取需要基於客戶端的狀態和環境來動態決定，同時企業也要能監控與衡量擁有資料資產的狀態。綜合上述來看，對於要提高 OAuth SSO 安全性，我們可以從提高身分驗證、調整授權令牌權限來改進，除了最初請使用者登入授權外，可額外針對使用者設備進行驗證，並透過登入週期作為一個單位對使用者進行設備驗證，而後每次使用者提出存取要求時，依據目前環境及設備狀況來動態決定。透過上述四點要求，企業能夠讓新興的存取方式，像 BYOD、雲端服務與遠端工作等能夠發展，並且防止資料外洩與限制內部橫向移動。



3.3 情境架構

以現今的企業來說，有提供雲服務、遠端工作、移動式個人設備(BYOD)等，本研究主要針對提供雲服務的企業進行考量，提出框架讓企業可確保使用者的設備安全性，使用者透過使用者裝置在進行應用服務存取時，會先進行設備綁定，在提出要求並通過設備驗證後，依據目前使用者設備情況來給予不同的風險等級。整體系統架構與概述會於後面進行詳細說明。

在企業 OAuth SSO 情境中，主要角色分別如下：

- 使用者：能夠授權應用服務存取使用者在身分提供者的資料，並透過 JWT 令牌存取應用服務。
- 應用服務：提供服務給使用者，透過 OAuth 存取令牌像身分提供者拿資料。
- 身分提供者：對使用者進行身分鑑別並確保使用者是否同意授權應用服務存取資料。



舉例來說，若企業使用雲端服務，以往使用者可直接登入並使用授權令牌獲取其需要資源，企業僅能對使用者進行最小權限控管，讓其拿取所需要的最小資源，但如今攻擊者可使用多種手法來竊取或假冒使用者到企業雲端拿取資料，如若企業無法控管，可能會造成許多資料外洩。如果透過本系統，藉由驗證設備安全性及評估使用者相關內容資訊後，可確保使用者端的設備安全性及依據目前狀態給與動態的安全等級評估，如若今天有攻擊者竊取了使用者的存取令牌，在初始的設備驗證就會失敗，無法使用此存取令牌來進行相關企業服務。除此之外，我們可看到圖 17，企業的雲端服務可依據敏感等級進行區分，如使用者 A 的設備有安裝防毒軟體且有更新目前作業系統的最新版，就可使用在防火牆內最敏感的資料；如使用者 B 的設備無安裝防毒，經由判斷就會將其信賴程度評分為較低，僅能存取在外圍的資料。

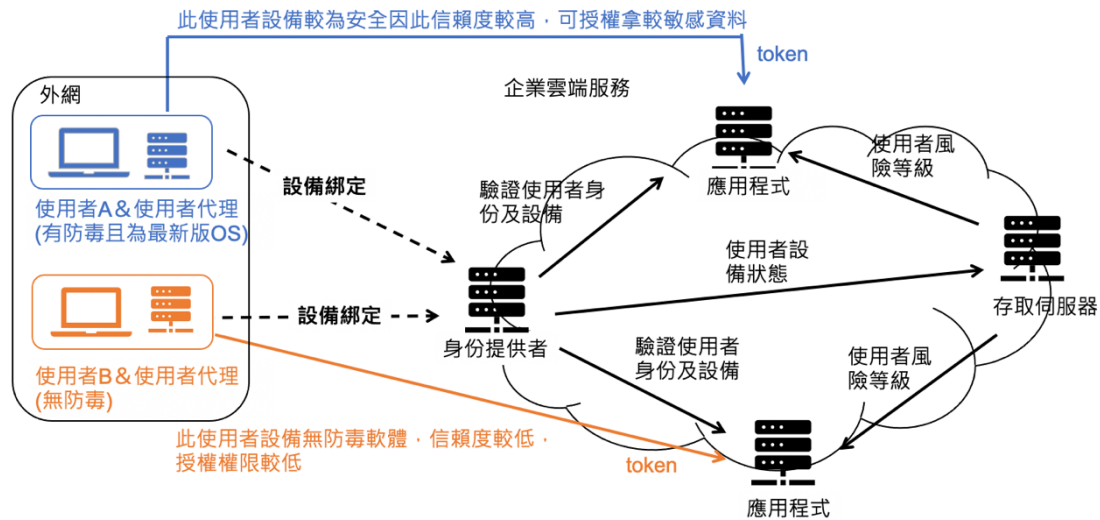


圖 17 使用情境
(資料來源：本研究整理)

以本研究整理的需求來看，使用者在使用應用服務前，身分提供者會與使用者進行設備綁定，在使用者提出存取要求應用程式時，應用程式會藉由存取令牌要求身分提供者對使用者做身分驗證及裝置確認服務，只要驗證成功後，使用者才可存取應用服務，滿足 R1 要對登入週期的資料存取進行要求許可。而應用程式透過身分提供者協助使用挑戰/回應的方式作為身分鑑別，滿足 R2 要確認送出要求的使用者為收到存取令牌的使用者，同時在身分提供者協助身分驗證過程中，透過使用者代理傳輸的設備狀況，可讓存取伺服器掌握設備狀態並對使用者設備安全性做出風險判定，動態決定使用者存取權限，滿足 R3，最後，企業藉由每次使用者要求存取了解使用者設備情況及存取權限的調整，能夠提升企業信賴區域內的資料安全性，滿足 R4。

第四章 框架概述與服務流程

4.1 框架概述

本論文提出的框架主要增加了設備管理模組，讓使用者在進行 OAuth-SSO 時，企業端可以驗證使用者身分及其設備，解決過往 OAuth 因只驗證存取令牌，導致令牌被偷竊時濫用的問題，同時在如今因辦公模式改變的情況下，讓企業能依據使用者及設備的安全性動態調整授權權限。

零信賴架構對於企業來說非常重要，因應現今網路邊界的模糊化，各種企業提供的新舊應用服務有可能因為防護與存取控制的弱點而被攻擊者竊取大量機敏資訊，在這樣大大小小的資安事件發生的情況下，零信賴架構的概念可對企業服務增加額外的防護。以零信賴原則來看，企業想建置零信賴架構時，需要在每次連線時，驗證使用者身分、評估設備安全性、訂定訪問應用程式的權限，我們要確保擁有訪問企業資源使用者的相關訊息，及了解使用者所使用設備的相關資訊，要能夠正確的驗證設備狀態，同時依據最小權限原則，讓使用者在通過身分驗證後，僅能獲得能完成工作內容所需的最低權限。

整體的系統框架如圖 18，角色主要有四個，分別為使用者、身分提供者、應用服務伺服器、存取伺服器，正常的 OAuth SSO 角色只有三個，在使用者同意授權後，應用服務會藉由 OAuth 存取令牌像身分提供者換取使用者鑑別資料，而使用者透過 JWT 令牌來存取應用服務。但在本論文框架中，除了原本的三個角色，論文提出了存取伺服器，主要用來評估使用者設備風險並動態決定授權權限。而其餘三個角色除了原本角色定義外，會基於 OAuth 協定讓身分提供者及應用程式能夠藉由 OAuth Token 來驗證使用者設備及使用者身分，確保使用者有存取權限，我們可藉由下表來了解框架角色定義。

本論文框架主要基於 OAuth 提出支援零信賴原則應用程式的存取方法，使用者要由不信賴區域進到企業信賴區域中要先進行設備綁定及驗證，而後使用者後續要存取應用程式時，應用程式會透過 OAuth 存取令牌要求身分提供者通知使用者應用程式隨機選擇的一個數值(challenge)，使用者可透過此隨機數作為存取應用服務所需要的 JWT 令牌的變動值，傳輸給應用服務進行認證，來確保是使用者本人，而使用者要提出存取要求時，也會透過使用者代理評估自身設備狀況藉由身分提供者傳給存取伺服器，讓應用程式可動態調整使用者存取權限。

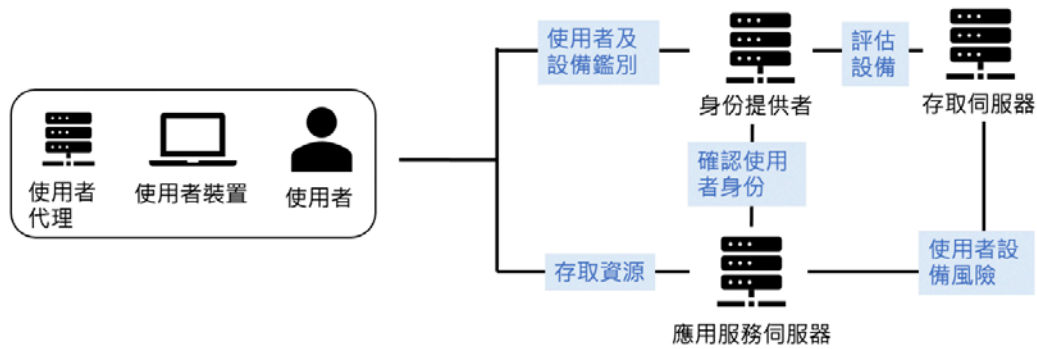


圖 18 系統框架
(資料來源：本研究整理)

表 3 框架角色定義

角色	定義
使用者	● 進行設備綁定才能在提要求時進行設備驗證 ● 用 OAuth 同意授權應用服務存取個人資料 ● 使用 JWT Token 存取應用服務 ● 透過使用者代理了解使用者設備狀況
身分提供者	● 基於 OAuth 協定的裝置確認服務 ● 協助應用服務使用挑戰/回應方式辨別使用者 ● 協助傳送使用者裝置資訊
應用服務伺服器	● 使用者同意後，運用 OAuth 取得使用者裝置的相關資訊 ● 透過挑戰/回應方式，在使用者存取服務時進行身分鑑別
存取伺服器	● 透過身分提供者所傳使用者裝置狀況評估使用者設備風險

(資料來源：本研究整理)

4.2 服務流程

我們可藉圖 19 概觀了解整個框架的運作模式，使用者使用應用服務時會先判定是哪個使用者，如使用者從沒綁定過，就不會有使用者相關資料，就須進行綁定後才能提出要求。當應用程式知曉是哪個使用者要存取服務時，就會用 OAuth Token 要求身分提供者通知使用者隨機數(challenge)，而身分提供者此時會先對使用者進行身分及裝置認證，如驗證成功就提供使用者隨機數，並讓使用者可透過此隨機數產生 JWT Token 存取應用程式，同時也會依據使用者此次要求的設備狀況更改使用者存取權限。要注意的是，驗證使用者身分及設備僅會在登入週期開始時進行，後續則是透過隨機數進行使用者身分鑑別。

現今企業因辦公需求改變導致 IT 架構變的複雜，因此新環境需要以零信賴架構為主，與以往只單純驗證使用者身分不同，要額外加強身分認證，將過往企業網路邊界進行強化，讓新興存取方式能夠支援零信賴架構原則，能夠防止資料外洩且限制攻擊者突破邊界後的橫向移動。透過此圖我們可以了解本研究重新設計身分認證，由使用者一開始要使用應用服務時的登入身分驗證，後續要存取資源時的存取控制，及每次依照使用者設備狀況進行評估來決定存取權限，本研究透過加入設備綁定、持續驗證及基於風險評估的動態權限調整來加強企業的存取控制，讓使用 OAuth SSO 的企業能夠支援零信賴架構。

圖 20 呈現的是整個框架架構圖，透過此圖可了解整個框架的運行流程及四個角色的定義，此圖僅是概觀讓我們了解整體流程，後續會在針對流程階段詳細定義並做仔細介紹。一開始我們會先進行使用者設備與身分提供者的綁定，且應用程式已獲得使用者授權取得 OAuth Token，同時使用者也知道所有他同意應用程式的公鑰，在這樣的情況下，當使用者登入時，由身分提供者進行使用者身分鑑別與確認使用者裝置，後續提出要求時，應用程式可基於 OAuth 取得使用者角色，並透過 OAuth Token 請求身分提供者提供基於 OAuth 協定的裝置確認服務，而後在使用 OAuth Token 向存取伺服器確認使用者存取權限。

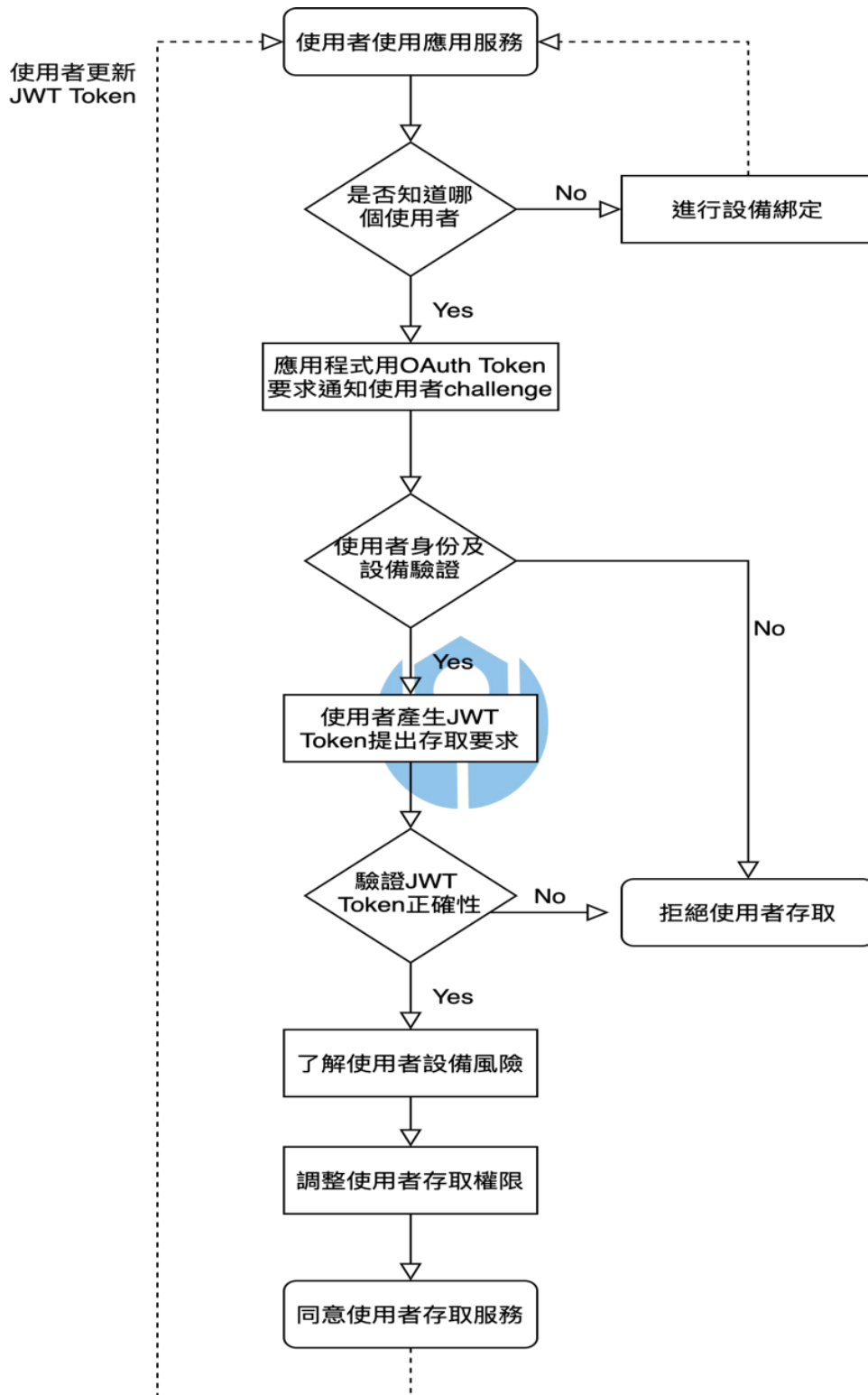


圖 19 框架作業流程圖
(資料來源：本研究整理)

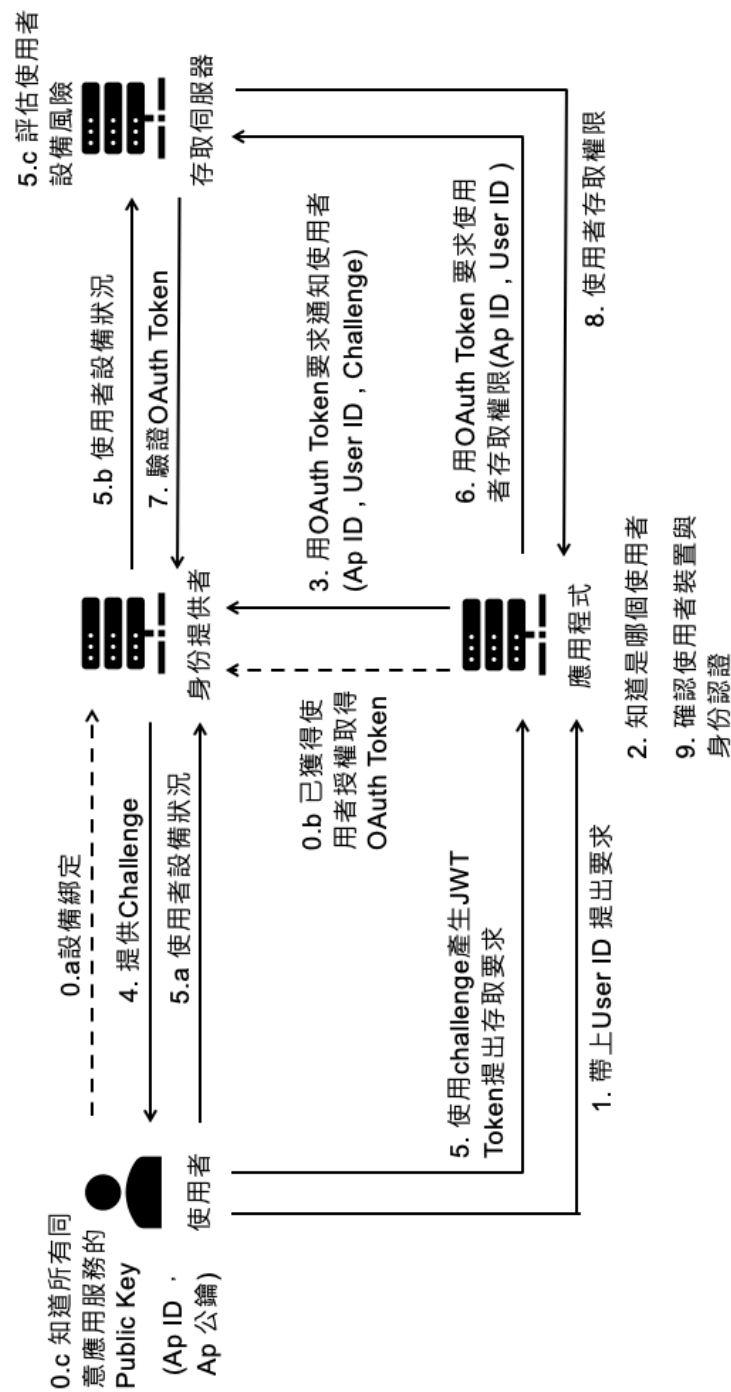


圖 20 框架步驟圖
(資料來源：本研究整理)

整個流程我們可將其分為兩階段，分別為初始的綁定階段及後續進行存取要求的階段，初次使用 OAuth SSO 需先進行設備綁定，而後以登入週期來做設備驗證，如同零信賴架構原則裡面所提到的，要將每一個主體視為不可信賴，每次連線都需進行驗證才能開始進行後續流程，以本研究來說，在存取要求階段內，要先驗證使用者及裝置，成功後，才能開始存取應用服務。而除了這兩階段外，使用者如果想更換設備也可針對綁定設備進行註銷，重新綁定設備。

初始綁定階段：使用者與身分提供者進行設備綁定。

存取要求階段：登入週期開始時要由身分提供者進行使用者身分及裝置鑑別，後續使用者才可要求存取應用程式，存取時會依據使用者設備判斷存取風險等級。

註銷階段：使用者可註銷綁定設備。

透過在 OAuth 原本授權流程中多加了設備綁定，在每次使用者提出要求時，多加一層防護，能夠確定提出要求的是否為當初拿取存取令牌的使用者，對於使用者及使用者設備做驗證，同時，透過了解設備即時狀態，動態更改使用者能獲得的權限範圍，也能讓企業對於內部的機敏資料多一層防護。以現今系統來說，許多系統有提供 OAuth SSO，本研究在基於 OAuth SSO 上重新設計了身分驗證機制，原本的存取控制是在使用者登入後與企業端後面的身分提供者進行身分鑑別，而本研究將應用服務與身分提供者整合起來，讓應用服務伺服器附加存取控制，在使用者提出登入或存取要求時，應用服務可直接藉由 OAuth Token 要求身分提供者進行使用者及使用者設備驗證，同時也能夠檢查使用者設備是否有符合安全性要求來更改存取權限。

在系統傳輸過程中，會有設備令牌、設備狀態聲明和應用程式存取令牌等敏感資訊，設備令牌為使用者設備與身分提供者做設備綁定時的參數，透過使用者設備的一些資訊產生一組設備令牌，傳送給身分提供者儲存，每次要進行連線時，透過此設備令牌進行設備驗證；設備狀態聲明則是藉由幾個判斷點來說明設備狀態，例如是否有安裝防毒軟體或是否防毒軟體是否有定期更新等，透過判斷點來了解設備狀態後產生狀態聲明傳送給存取保護伺服器並轉傳給政策引擎，來判斷設備安全性等級；應用程式存取令牌則是使用者要存取應用服務時的參數，每次存取後此參數都會進行變動來確保驗證使用者身分。上述所說的幾個參數都是在傳輸過程中的敏感資訊，從使用者端的不信賴區域傳輸到信賴區域中間可能會有許多攻擊，因此我們需要確保傳輸過程的安全，系統會對這些參數進行加解密及

簽章，整個加解密流程所使用的參數如表 4，接下來分別會依據各阶段的流程進行說明。

表 4 使用參數

參數	說明
(p, a, b, g, n, h)	橢圓曲線公開參數
U_p	使用者公鑰($U_s = U_p G$)
U_s	使用者私鑰
I_p	身分提供者公鑰($I_s = I_p G$)
I_s	身分提供者私鑰
A_p	應用程式公鑰($A_s = A_p G$)
A_s	應用程式私鑰
SK	共享秘密
K_{ENC}	對稱式金鑰
K_{MAC}	訊息簽章金鑰
$E(\cdot)$	對稱式金鑰加密
$D(\cdot)$	對稱式金鑰解密
$SIGN(\cdot)$	對訊息簽章
tag	簽章後的資料

(資料來源：本研究整理)

4.2.1 初始綁定階段

企業使用 OAuth-SSO 時，使用者都需要先註冊，如同一般第三方應用程式想要授權獲取使用者資料時，都需要先到企業進行註冊並獲取 Client Identifier 和 Client Secret，進行註冊主要是為了獲取唯一的識別號，這樣後續要求存取代碼時，授權伺服器才能識別不同使用者的身分。在註冊過後，使用者就可至企業下載使用者代理，此使用者代理主要有幾個功能：產出設備令牌、進行環境檢測，在初始綁定階段時主要會產出設備令牌並與身分提供者進行設備綁定。

企業一般會使用 IAM(Identity and Access Management)識別使用者身分，主要是會依據三種驗證因素來評估：使用者知道的內容、使用者擁有的內容、使用者本身的內容，就像是使用者密碼、權杖、使用者指紋等等來進行驗證。就本系統來說，可將設備視為使用者，使用設備所擁有的內容來進行設備的身分驗證，因此使用者代理會在初次啟動時產出設備令牌作為設備身分識別的驗證因素。

設備令牌主要參數說明可參考表 6，而主要格式為：


$$device\ token = H(client\ id \parallel device\ id)$$

表 5 設備令牌參數

參數	格式	說明
Client ID	String	註冊使用服務的使用者唯一標示符
Device ID	String	使用者設備的唯一標示符

(資料來源：本研究整理)

考量到使用者有多種設備會進行綁定，透過使用者 ID 及設備 ID 能夠讓使用者產出專屬設備的設備令牌，讓使用者可綁定多台設備，而傳輸設備令牌過程中，為了確保在不信賴區域中可以安全地保護設備令牌，因此使用使用者及身分提供者的公私鑰對進行加密及簽章，同時會進行雙向鑑別來防止中間人攻擊。

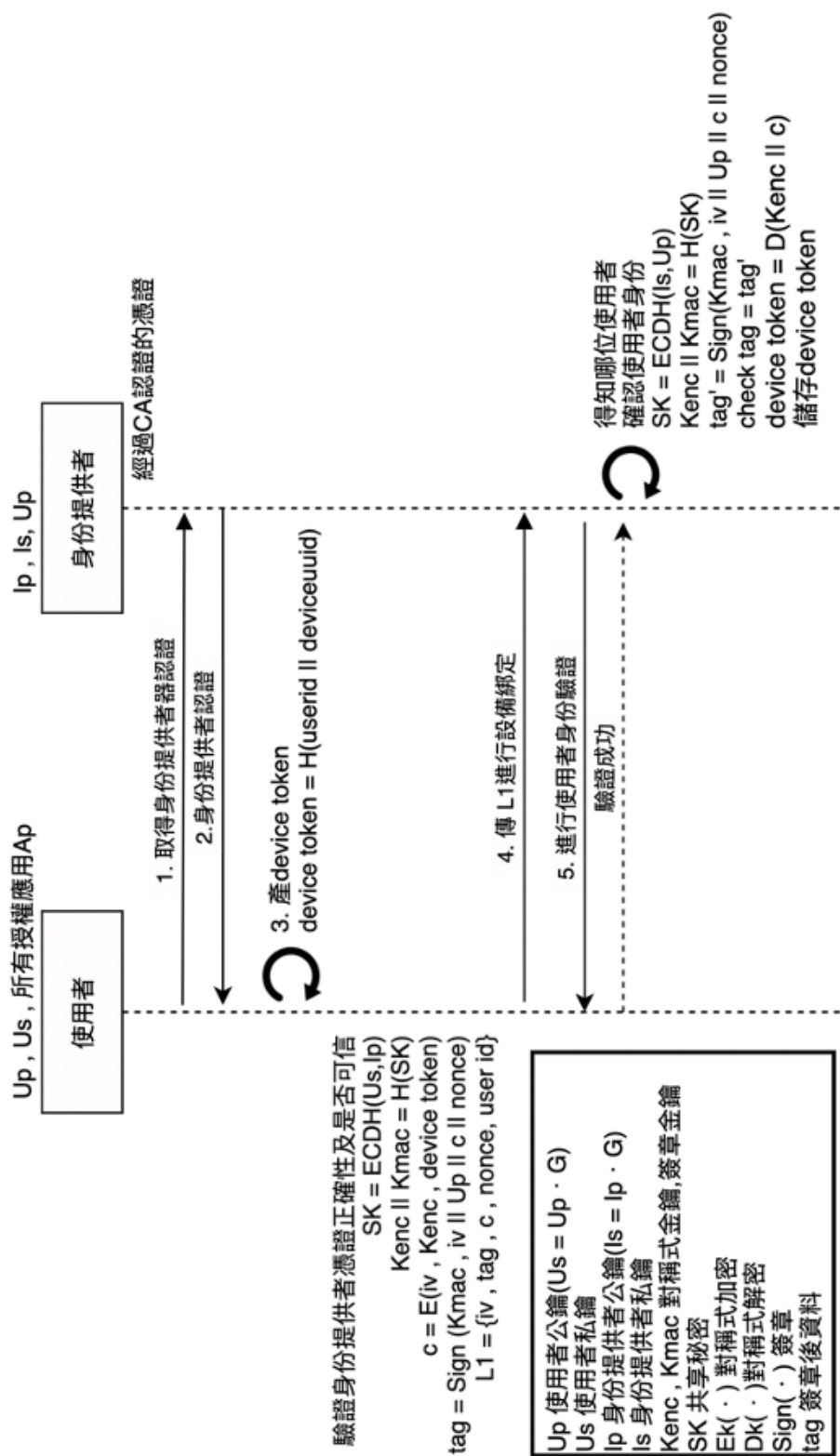


圖 21 初始階段流程

(資料來源：本研究整理)

初始綁定流程步驟：

1. 使用者取得身分提供者憑證
2. 使用者代理透過使用者 ID 及設備 UUID 產生專屬設備的設備令牌
3. 驗證身分提供者憑證正確性後，將設備令牌進行加密、簽章，並傳輸給身分提供者
4. 身分提供者收到後，對使用者進行身分驗證，確認是使用者本人
5. 了解是哪位使用者並解密設備令牌，將使用者 ID 及設備令牌儲存



4.2.2 存取要求階段

在設備綁定成功後，當使用者提出要求時，會視為一個登入週期的開始，應用程式會藉由身分提供者來進行使用者身分及設備驗證，確保使用者為正確使用者，同時應用程式會透過 OAuth Token 要求身分提供者通知使用者 challenge，在身分提供者驗證好使用者身分後，將 challenge 傳給使用者，使用者會透過此 challenge 及序號產生 JWT Token 來對應用程式提出存取要求，應用程式這時可再透過 OAuth Token 對存取伺服器了解使用者設備風險等級，在驗證 JWT Token 正確性後，讓使用者能夠存取使用者對應的存取權限等級。只要使用者還在登入週期內，會視為使用者還是可信賴，僅會藉由使用者的動態 JWT Token 及每次要求的設備狀況進行評估及更改存取權限，若登入週期過了，使用者就需要再進行身分及設備驗證，重新開始存取要求階段流程。

存取要求階段的過程，主要有設備驗證、JWT Token 驗證、設備狀態風險評估，設備驗證透過傳遞(設備令牌 || nonce)簽章及 nonce，在不洩漏設備令牌的情況下讓身分提供者也可以驗證簽章的正確性；JWT Token 及設備風險評估主要是使用者存取應用程式的存取令牌及決定使用者授權權限參考值，應用程式可基於 OAuth Token 像身分提供者及存取伺服器確認使用者裝置及使用者存取權限，前述 JWT Token 主要的變動值來自應用程式提出的 challenge，每次登入週期內透過此 challenge 及序號作為登入週期內要求次數的鑑別，只要登入週期結束，重新登入後，challenge 就會改變。設備風險評估是透過使用者代理產生設備狀態聲明來進行評估依據，使用者對應用程式提出要求時會順帶將設備狀態聲明提供給身分提供者，請它傳給存取伺服器進行評估。

在階段過程中，可以看出以登入週期為單位進行使用者身分及設備驗證，能夠支援零信賴架構原則每次存取皆需要驗證身分，後續透過使用者要求時，使用者代理監控使用者設備產出的設備狀態聲明，動態調整使用者存取權限，維持企業資料安全性，我們可透過下圖來了解存取要求階段流程。

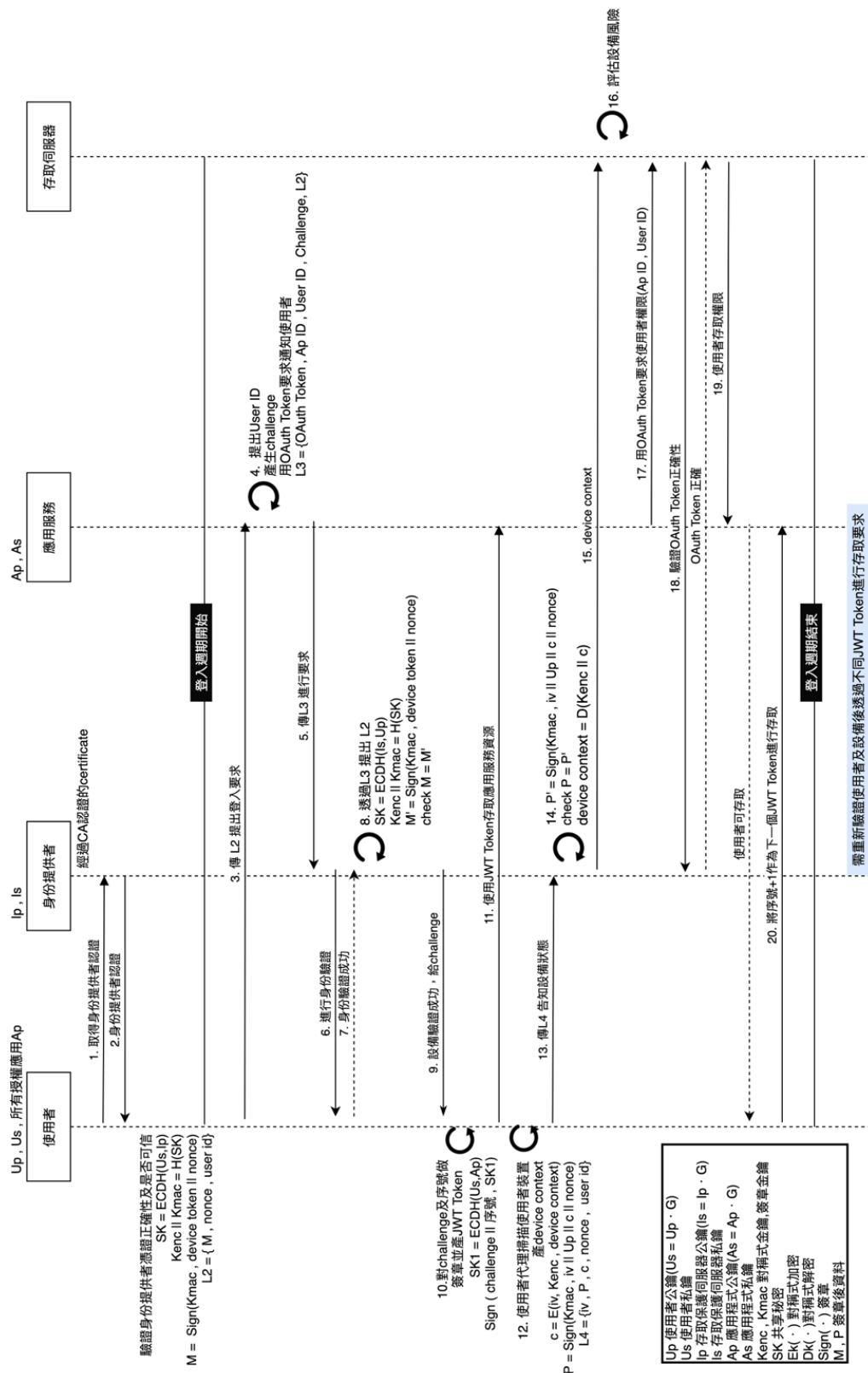


圖 22 存取要求階段流程
(資料來源：本研究整理)

存取要求階段流程步驟：

1. 使用者取得身分提供者憑證
2. 使用者代理對(設備令牌 || nonce)簽章並傳輸 User ID、簽章、nonce 對應用服務提出登入要求
3. 應用服務透過 User ID 得知是哪個使用者，使用 OAuth Token 要求身分提供者將 challenge 給使用者
4. 身分提供者對使用者進行使用者身分驗證及裝置鑑別來確保使用者後，將 challenge 提供給使用者
5. 使用者使用此 challenge 及序號產生 JWT Token，並透過使用者代理產生使用者設備狀態聲明，將 JWT Token 傳給應用服務，設備狀態聲明加密後傳給身分提供者
6. 身分提供者透過解密提出設備狀態聲明，傳給存取伺服器
7. 存取伺服器透過設備狀態聲明評估使用者設備風險，決定使用者存取權限
8. 應用程式透過 OAuth Token 向存取伺服器要求使用者存取權限，存取伺服器藉由身分提供者驗證 OAuth Token 正確性後，將使用者權限給予應用服務
9. 應用服務持有 JWT Token 及使用者存取權限，同意使用者可存取

設備令牌聲明是每次連線時使用者代理透過檢測目前設備的環境來像企業說明設備的狀態，就像民眾要使用自然人憑證時，會需要做環境檢測，了解民眾使用的作業系統及瀏覽器版本、是否有安裝 HiCos 元件及相關驅動程式，做好基本檢測後，民眾才可使用。本系統會依據設備狀態來決定存取敏感資訊的範圍，

假設此設備目前作業系統為最新版本且有安裝防毒軟體，企業就能給予此使用者較高的權限；如此設備雖有安裝作業系統最新的補丁，但無安裝防毒軟體，為了確保企業內部資源的安全，企業只能給予此使用者信任度等級較低的權限。

本論文主要會針對以下 8 項了解設備狀態，此八項主要由本研究自國內外文獻整理[16][17][18][19]：

- 目前作業系統版本
- 是否有使用防火牆
- 是否有使用主機入侵檢測系統
- 是否有裝防毒軟體
- 防毒軟體是否有定期更新
- 是否有安裝垃圾郵件過濾器
- 是否有使用跳出視窗阻擋器
- 是否有關閉沒使用的埠



在掃描設備狀態後，使用者代理會用 Base64 的二進位資料表示法來呈現目前設備狀態，之後將其傳給身分提供者，再由身分提供者傳給存取伺服器，存取伺服器主要會根據我們所訂定的風險清單來做為風險評估的指標，以設備的作業系統為例，可以依照作業系統是否通過安全評估共通準則(Common Criteria, CC)內的安全評估保證等級(Evaluation Assurance Levels, EALs)第四級[16]，也就是 EAL 4，安全評估共通準則總共以七個安全評估等級來訂定產品的安全性，而針對商用產品的最高等級為 4，因此可以將此作為判斷依據，下表為系統判斷設備的安全性的依據，主要以 8 項檢查項目來判斷設備安全，8 個項目分別擁有不同的信賴分數，將設備目前有達成的項目進行信賴分數加總，並依據閾值來訂定設備安全性的高中低。

表 6 判斷設備的安全性的依據

檢查項目	說明	信賴分數
作業系統版本	● 作業系統版本是否通過 CC EAL 4 級或以上認證 ● 作業系統版本是否為最新版	30
是否有使用防火牆	● 防火牆為防禦的第一道牆	20
是否有使用主機入侵檢測系統	● 可查驗系統事件、日誌	5
是否有裝防毒軟體	● 可防止惡意軟體	20
防毒軟體是否有定期更新	● 確保防範最新惡意程式	10
是否有安裝垃圾郵件過濾器	● 防止收到釣魚信件導致電腦中毒	5
是否有使用跳出視窗阻擋器	● 減少系統執行不必要程式	5
是否有關閉沒使用的埠	● 未使用的埠可能會被攻擊者入侵	5
設備安全性高：最終信任分數為 90 以上 設備安全性中：最終信任分數為 70 以上 設備安全性低：最終信任分數為 70 以下		

(資料來源：本研究整理)

當存取伺服器經由設備狀態聲明來評估設備安全性等級的高中低後，便會在應用程式要求使用者權限等級時將結果傳送給應用程式，讓應用程式以此作為授權範圍的判斷依據。與 OAuth 過往流程相比，主要多了動態調整風險的功能，依據企業內受保護資料等級、使用者要求等級與設備安全性等級來作為使用者存取權限的最終判定，整體考量點會以最小權限原則來訂定，可參考表 7。

表 7 最終給予受保護資料等級比較表

	受保護資料機敏等級	使用者要求等級	設備安全等級	最終決定等級
1	高			
	中	V	V	V
	低			
2	高	V		
	中		V	V
	低			
3	高		V	
	中			
	低	V		V

(資料來源：本研究整理)

4.2.3 註銷階段

註銷階段為獨立出來的一個系統流程，使用者設備若有問題需汰換或維修等問題，為了防止設備不在身邊所造成的困擾，系統提供註銷設備的服務，可以移除設備的存取權。這樣一來，身分提供者就無對應的設備令牌，不論設備是否被惡意使用者發現並使用，都無法通過設備驗證，詳細流程可參考圖 23。當使用者提出註銷要求時，一樣要先經過驗證階段，方可進行註銷；若使用者設備遺失，也可重新綁定設備後，再註銷原設備。

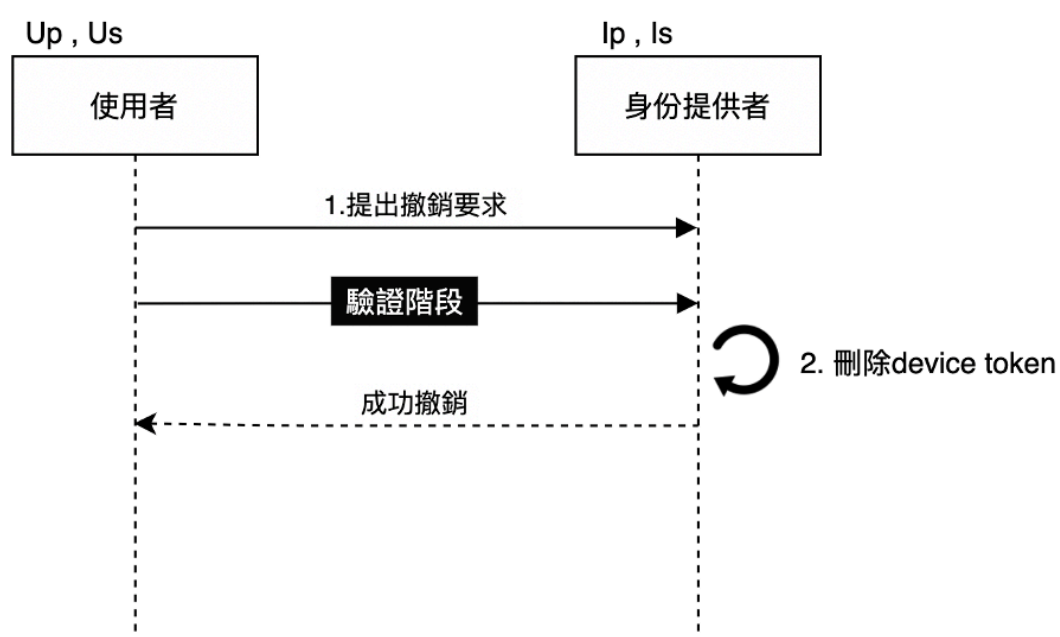


圖 23 註銷流程
(資料來源：本研究整理)

第五章 實作與分析

5.1 系統實作環境

本研究將以 windows 10 為作業系統，硬體規格如下所述，本研究主要是以 OAuth2 授權流程為基礎，提出方法讓傳統以令牌為鑑別方式能支援零信賴原則，能夠將使用者設備進行綁定並在每次連線時進行驗證、判斷風險等級，因此會以本研究提出的方法與傳統 OAuth 授權流程做比對，了解本研究方法效能如何。

1. 作業系統：

Windows 10

2. 處理器：

Intel(R) Core(TM) i7-9700 CPU @ 3.00GHZ



3. 記憶體：

32GB

4. 儲存硬碟：

INTEL SSDPEKKW512GB

5. 開發語言：

Python3.7

5.2 系統框架實作概述

論文實作主要以 Flask 框架之網頁伺服器及 Restful API 來完成，使用者使用時透過使用者代理將使用者裝置與身分提供者進行設備綁定，後續便可經由通過設備驗證進入企業的信賴區域來提出要求，而企業內部也會藉由判斷設備情況來決定授權權限，主要實作流程如圖 24 所示。

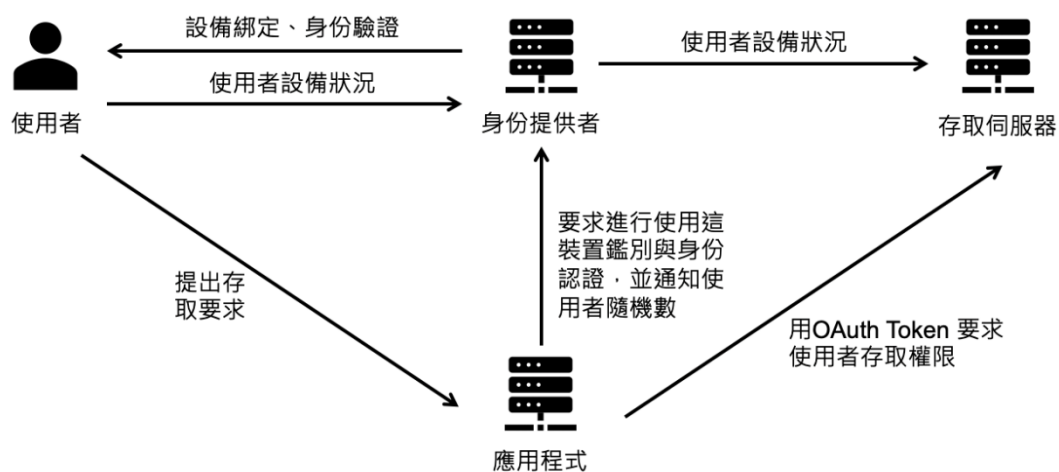


圖 24 框架實作流程
(資料來源：本研究整理)

在實作系統過程中，會先假設使用者設備已經有使用者代理，系統主要實作部分為 1.使用者透過使用者設備綁定設備並在綁定後提出要求 2.當使用者提出要求後，系統會進行經過設備驗證，並在通過後，讓使用者要求可進入內部信賴區域進行後去判斷。一開始進行設備綁定時，企業端的身分提供者不會有使用者設備的相關資料，因此需要先將使用者設備的使用者代理與使用者做對照，而後續驗證設備時，也需要確保提出要求的使用者與當初綁定的使用者為同一人，因此需要對設備令牌進行驗證，表 8 為設備綁定與設備驗證時系統所需使用到的參數資訊，當身分驗證後，進入到企業內部的隱式信賴區，系統會針對設備狀態的風險做評估，表 9 呈現設備風險評估與授權資料評估所需的參數資料。

表 8 設備綁定與驗證格式

服務	參數內容
綁定裝置服務	User ID: int Device Token: String
驗證設備服務	User ID: int Sign (Device token nonce) : String

(資料來源：本研究整理)

表 9 設備風險評估與授權資料格式

服務	參數內容
設備風險評估	User ID : int Device context: String
授權資料評估	User ID : int Ap ID: int OAuth Token: String Device Security Level: String

(資料來源：本研究整理)

```
def gen_devicetoken():
    osversion=str(platform.uname())
    location=get_userlocation()
    nowtime=str(time.time())
    ori_devicetoken=userid+"||"+deviceuuid
    devicetoken=ori_devicetoken.encode(encoding="utf-8")
    return devicetoken

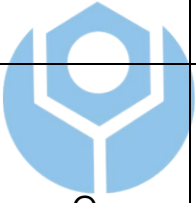
def encrypt():
    secp_k=generate_key()
    sk_bytes=secp_k.secret
    pk_bytes = secp_k.public_key.format(True)
    enc_devicetoken=encrypt(pk_bytes, devicetoken)
    return enc_devicetoken
```

圖 25 設備令牌產生及加密程式碼

(資料來源：本研究整理)

以本系統提出的方法來看，傳統以令牌為鑑別方式的 OAuth 或 OIDC，透過令牌雖可快速鑑別並授權，但無法確認使用令牌的使用者是否為攻擊者所偽造，即便 OIDC 後來基於 OAuth 流程之上加了使用者身分驗證，但對於第三方應用程式的身分驗證並無著墨，但透過本研究提出框架，可在使用令牌作為授權認證前，先進行使用者及設備的身分驗證，減少資料外洩的資安風險。

表 10 方法比較

	OAuth	OIDC	本研究所提方法	本研究如何達成
使用者鑑別方法	存取令牌	存取令牌及身分令牌	身分及裝置驗證	
R1：需要針對每次登入週期的資料存取進行要求許可	X	X	O	以登入週期為單位對提出要求使用者進行身分驗證及裝置識別
R2：要確保送出要求的客戶端為收到存取令牌的客戶	X	 O	O	應用程式基於 OAuth Token 對身分提供者要求向提出要求的使用者傳輸 challenge
R3：能夠確保客戶端設備的安全性並依據設備安全性提供不同等級的受保護資料	X	X	O	依據使用者代理監控使用者設備，並傳輸設備狀態聲明來對每次使用者連線狀況進行動態授權權限調整
R4：能收集客戶端設備狀況來增進安全狀況	X	X	O	透過每次使用者要求時了解使用者設備狀況，保護企業內部資源安全

(資料來源：本研究自行整理)

5.3 系統效能測試

為了讓傳統以令牌為基礎的鑑別方式能支援零信賴架構原則，本研究基於 OAuth 授權流程上做了些許變動，以 OAuth SSO 流程來說，使用者透過第三方應用程式對企業端應用程式提出要求，企業端在驗證使用者身分並獲得使用者授權後，便可發存取令牌讓使用者透過此令牌獲得相關應用程式使用資格或拿取相關機敏資料。為了解決過往授權流程中身分驗證不足的問題，本研究提出的框架會讓使用者在使用者設備上使用使用者代理來證明使用者設備身分，同時透過企業端的身分提供者驗證使用者身分，在還沒確認提出要求的人是否安全，不會讓其進入內網，另外對於提出要求的使用者設備，系統會針對設備狀態來評估並給予相對應的風險值，來降低資料外洩的風險。本研究雛形以 windows 10 為作業系統，詳細規格如 5.1 所示，在此雛形中，主要是使用基於 Python 的網站框架套件 Flask 來架設應用服務，其餘應用服務伺服器、身分提供者、存取服務器則是使用 Docker 來架設容器，將各容器之間環境做區隔來減少軟體版本之間衝突。

整體來說，本研究與 OAuth 流程差異總共有三個，(1)一開始使用者要登入時的設備綁定、(2)登入週期開始時的設備驗證及(3)對設備狀態做評估給定風險值，對於 OAuth 流程做最小的變動來解決過往身分驗證機制不足而有資料被竊的風險，同時又能夠支援零信賴架構原則。為了了解與 OAuth 流程的差異，本研究針對三個與 OAuth 流程不同的步驟統計平均時間，了解在以往 OAuth 流程的執行時間上，額外多出多少時間能解決身分驗證並動態調整設備所提出的授權權限，與 OAuth 流程不同的差異流程如果執行時間並不會差異過多，就能確保此系統的效能，既能解決身分驗證機制不足的問題，又與以往 OAuth 流程差異不大。

(1.) 設備綁定程序效能評估

設備綁定為針對使用者設備與企業端的身分提供者進行綁定，使用者透過使用者代理產出設備令牌並傳輸給身分提供者進行儲存，達到綁定效果，可參考圖 21 初始階段流程說明，使用者會使用 POST/provider/binding 對身分提供者進行設備綁定，首先會對身分提供者進行驗證，之後才會產出設備令牌傳輸給身分提供者進行綁定，整個過程會使用雙方公私鑰對及用 ECDH 協商的共享金鑰將設備令牌加密並傳輸。本研究透過模擬 100 組使用者設備產

出的設備令牌並與身分提供者進行綁定來看設備綁定的平均執行時間，如圖 26，執行 100 次設備綁定的平均時間為 0.0122 秒，設備綁定為使用者初次使用設備要進行要求時才需進行的，因此此時間與原本 OAuth 授權流程並不衝突，使用者可透過使用者代理與身分提供者的加密傳輸來保護設備令牌，而為了確保不會有攻擊者偽造使用者進行設備綁定，使用者及身分提供者會進行雙向鑑別來防止攻擊。

表 11 設備綁定 API 資訊

API	POST/provider/binding
參數	User ID Device Token
用途	使用者可對身分提供者提出設備綁定

(資料來源：本研究整理)

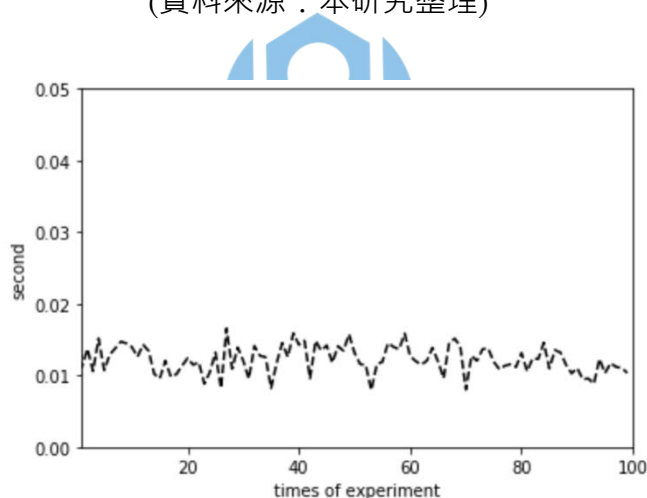


圖 26 單一設備執行 100 次的綁定時間圖

(資料來源：本研究整理)

(2.) 設備驗證程序效能評估

每次使用者初次登入時，會透過身分提供者進行身分及設備驗證，也就是提要求時要帶上使用者 ID、設備令牌簽章，藉由應用程式使用 POST/provider/{apId}/authentication_user 像身分提供者要求使用者身

分鑑別，之後透過身分提供者驗證設備令牌簽章正確性來證明使用者身分，驗證完使用者設備後，透過提供 challenge 來讓使用者動態產 JWT Token 存取應用程式，整個設備驗證流程可參考圖 22 存取要求階段流程前部分的驗證階段，與過往傳統的鑑別方式不同，使用者一開始在不受信賴的區域提出要求時，會先經過身分查核，才能進入可信賴區域，減少企業資料外洩風險，也因為多了設備驗證這一道關卡，與以往直接傳遞存取令牌要求資料不一樣，因此本研究透過執行 100 次設備驗證，來了解設備驗證的平均執行時間為何，可看到圖 27，基本平均時間落在 0.1 至 0.2 之間，執行 100 次下來的平均時間為 0.0102 秒，對於要求的使用者來說並不會造成效能的影響。

表 12 設備驗證 API 資訊

API	POST/provider/{apId}/authentication_user
參數	Application ID User ID Challenge
用途	應用程式對身分提供者要求使用者身份驗證

(資料來源：本研究整理)

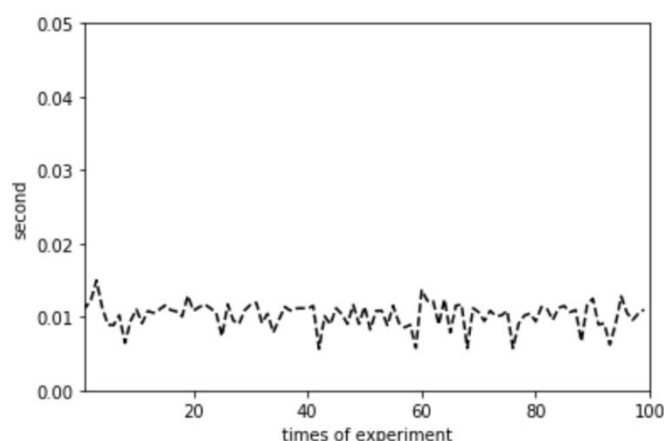


圖 27 單一設備執行 100 次的驗證時間圖

(資料來源：本研究整理)

(3.) 風險評估程序效能評估

在前述步驟驗證設備身分後，便會將使用者設備資訊傳給存取伺服器進行使用者設備安全性判斷，依據 8 項評估標準來了解此次連線的設備狀況，依據設備的情況來給予信賴分數，並在最後評估此設備安全性為何，評估完成後，將安全性等級傳遞給應用程式，讓應用程式依照使用者提出要求的權限等級與使用者設備安全性等級做比較，並依照比較結果決定給予使用者要求的資料等級，並傳回給使用者，比較過程系統採取最小權限原則，不論哪一方等級低於另一方，就會依照最低的權限給予使用者資料，防止資料外洩。而依據使用者要求等級與設備安全等級做比較所給予的資料等級權限不一樣，本研究隨機模擬 100 組授權要求，透過圖 28 呈現評估風險的執行時間，模擬 100 次由存取保護伺服器將要求送至存取伺服器評估及應用程式授權後給予使用者資料的秒數為 0.0105 秒。

表 13 評估風險 API 資訊

API	POST/service /{apId}/user_risklevel
參數	Application ID User ID
用途	應用程式對存取伺服器要求使用者風險等級

(資料來源：本研究整理)

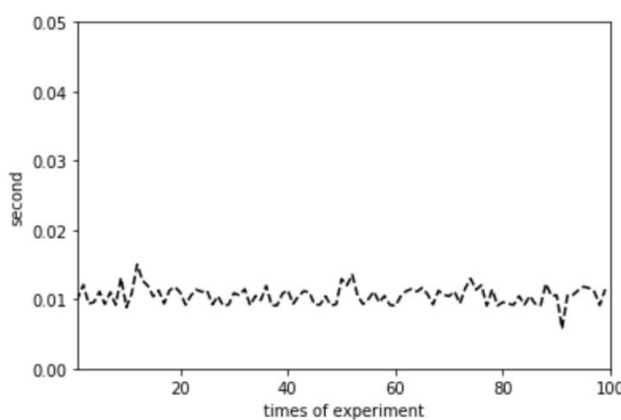


圖 28 單一設備執行 100 次的評估風險時間圖

(資料來源：本研究整理)

總結三者差異來看，我們撇除初次使用須進行的設備綁定，與以往以令牌作為鑑別方式的 pure OAuth 授權來比較，本研究模擬 100 次過往 pure OAuth 授權方式所需時間，其平均執行時間為 0.007 秒，而本研究的方法在進行授權時，需先進行設備驗證與評估風險，兩者也模擬 100 次執行時間，分別為 0.0102 秒與 0.0105 秒，將兩者所需時間加總並與 pure OAuth 授權流程時間相比，差異如 29 呈現，本研究所提出的新授權方法與以往授權方法差異為 0.013 秒，並不會對提出要求的使用者造成影響，但透過設備綁定、驗證設備身分與動態評估風險卻能讓企業與使用者都減少對資料外洩的風險疑慮。

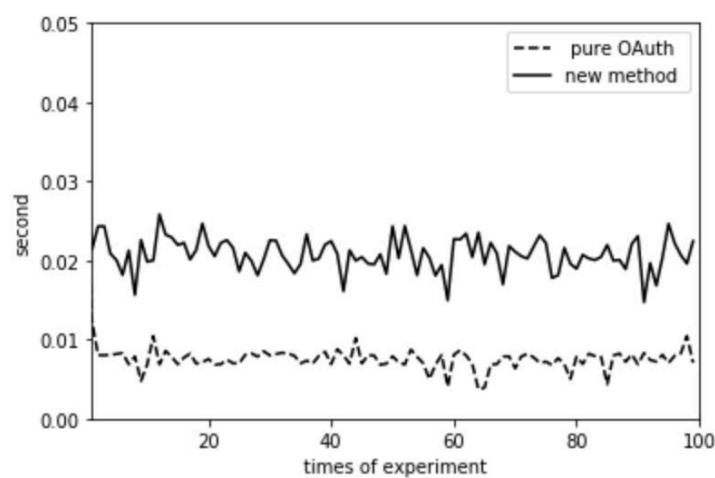


圖 29 兩者方法差異時間比較
(資料來源：本研究整理)

5.4 系統安全性分析

針對讓本論文所提出讓傳統以令牌為基礎的鑑別方式能夠支援零信賴架構原則的方法，我們需要特別注意其安全議題，因傳統的令牌鑑別方法會有不可避免的資安問題，在本研究提出改善方法後，需要驗證本研究所提出的系統是否安全，本研究採用微軟提出的 STRIDE 威脅模型來評估使用者端、企業端之間的資安議題，此威脅模型總共分為六種類型，分別為欺騙(Spoofing)、竄改(Tampering)、否認(Repudiation)、資訊洩露(Information disclosure)、拒絕服務(Denial of Service)、特權提升(Elevation of privilege)，在透過 STRIDE 威脅模型分析後，從威脅模型當中針對論文提出框架流程談討攻擊者手法進行探討。因本論文情境主要為使用者存取企業應用程式，企業應用程式要確保使用者身分及使用者設備安全性來進行授權權限調整，在探討攻擊手法時，主要會以使用者端為考量點。同時在第三章的需求分析中有提到在考量 OAuth SSO 及零信賴原則下，本研究有提出 R1 到 R4 的四點要求，在本章節也會針對如何達成此四點要求進行說明。



(1.) 欺騙(Spoofing)

欺騙是攻擊者可透過假冒使用者身份來讓伺服器誤給權限或機敏資料，為了要防範此類型攻擊，最好的作法就是進行身份驗證，在本系統中，會透過設備綁定及設備驗證來讓身分提供者端確保使用者身份，即使攻擊者真的獲取了存取令牌或授權代碼，在偽冒使用者提出要求時，身分提供者便可藉由設備身份驗證得知提出要求的並非真實的使用者，就可拒絕其要求，系統主要是藉由設備令牌進行粗粒度的身份識別，透過在最外層進行設備驗證就可阻擋許多偽冒使用者身份的攻擊者。

(2.) 竄改(Tampering)

竄改為改變資料造成資料污損，在使用者未授權的情況下針對資料進行改動，破壞資料完整性，為了要防止竄改攻擊，我們需要在資源改動時進行相對的身份驗證，確保在資源變更時是否擁有相關授權同意。攻擊者

可透過存取要求的過程中竄改存取資料的使用者或更改想存取的資料，因此本系統需透過持續驗證的方式來防止此類型的攻擊手法發生。

(3.) 否認(Repudiation)

否認為使用者進行某項行為後，卻否認自身曾經做過，類似隱匿蹤跡的行為，因此伺服器端需要在使用者進行任何行為或存取資源時，進行記錄，讓使用者無法否認，系統會在使用者進行資源存取時，透過令牌辨識使用者身份，並將使用者操作紀錄額外存於資料庫中，確保每位使用者的每次操作記錄都會有有效記錄，達成不可否認性。

(4.) 資訊洩露(Information disclosure)

資訊洩露為讓未授權的人能夠存取敏感資訊，導致資料外洩，以使用者角度來看，如果沒有做好身份驗證，有可能造成他人能夠訪問使用者隱私資料，若要維護個人隱私資料的保密性，系統需做好控制措施，要確保他人不可訪問非個人的機敏資料，本系統在使用者提出要求時，先針對設備做粗粒度設備驗證，後續會在透過令牌進行細粒度權限判別，因此能夠阻擋攻擊者對於非本人的資料要求，同時還能夠對資料的權限訪問有一定的控管要求，達成資料的保密性。

(5.) 拒絕服務(Denial of Service)

拒絕服務是讓資源耗竭而無法繼續提供服務，不論是對於使用者或伺服器端，若沒有對系統進行全面控管，若有攻擊者試圖提出大量要求造成系統癱瘓，降低系統的可用性，就會造成使用者無法順利使用該服務，因此可於本系統中加入連線控管，對於不信賴區域中異常連線的要求進行黑名單控制，讓其無法造成系統可用性問題。

(6.) 特權提升(Elevation of privilege)

特權提升是攻擊者透過違法手段來讓本來不具有特權的使用者能夠獲得特權存取，為了要防止此類攻擊，要警慎的規範所有人員使用內部資源的權限，而在本系統中，對於提出要求的使用者，系統也會針對使用者設

備當次連線的狀況進行評估，假如使用者設備狀況不錯，就可給予使用者最小權限的資源，但如果該次狀態不佳，會降低使用者所能獲得的權限，來減少惡意手段獲取特權的機率。

透過上述 STRIDE 威脅模型分析，我們可以知道在整個系統流程運行過程中，攻擊者可以透過欺騙、竄改、特權提升等攻擊手法來試圖獲得存取權限，如透過偽冒身分來欺騙使用者並趁機竊取使用者敏感資訊藉此獲得企業內部資源，或是透過擷取傳輸過程中的資訊加以重送，讓企業端以為是使用者送來的存取需求並且給予回應，除此之外，攻擊者也可透過遺失的使用者裝置來進行設備驗證，讓身分提供者驗證使用者身分，因此，在經過威脅模型分析之後，本研究針對分析過後的結果進行雛形框架攻擊手法的探討，如表 14。

表 14 圖 20 中各步驟資料傳輸說明及攻擊手法探討

步驟	資料傳輸方	資料接收方	傳輸資料	攻擊手法探討
步驟 1	使用者	應用程式	提出要求 (User ID)	攻擊者可重送此要求企圖獲取隨機值
步驟 2~3	應用程式	身分提供者	OAuth Token App ID User ID Challenge	雙方於企業內網傳輸，可確保資料不外洩
步驟 4	身分提供者	使用者	Challenge	攻擊者可偽造身分，使用中間人攻擊
步驟 5	使用者	應用程式	存取要求 (JWT Token)	攻擊者可藉由重送或中間人攻擊來獲取敏感資料
步驟 5.a	使用者	身分提供者	使用者設備狀況	攻擊者可竄改使用者設備情況來藉此提權
步驟 6~9	應用程式	存取伺服器	OAuth Token App ID User ID	雙方於企業內網傳輸，可確保資料不外洩

(資料來源：本研究整理)

藉由上表我們可以得知在本研究雛形框架中各步驟所傳輸的資料，並藉此了解框架的威脅模型，從中知曉在各個步驟中攻擊者可能使用的手法，接著我們針對這些攻擊手法進行探討，以攻擊者角度出發來了解本研究提出的框架是否能預防這些攻擊。

(1.) 重送攻擊：攻擊者竊取使用者要求，並將要求重送給應用程式。

- 攻擊者竊取「使用者登入要求」

攻擊手法：攻擊者在一開始使用者提出登入要求時，竊取此要求，並對應用程式提出帶上 User ID 的登入要求，希望能獲取 challenge 來提出存取要求。

防禦方法：當應用程式收到攻擊者傳送的登入要求，辨別是哪個使用者後，會藉由 OAuth Token 像身分提供者提出驗證使用者身分的要求，在身分提供者驗證後在提供 challenge，由於使用者與身分提供者事先有綁定過，即時是攻擊者提出登入要求，身分提供者也是與正確的使用者進行身分驗證並提供 challenge，攻擊者無法獲取，因此不會造成資安風險。而在這個階段中，因為有事先進行設備綁定，能夠確保是正確的使用者獲取應用程式提供的 challenge，能夠滿足 R2 要求，確保送出要求的使用者就是收到令牌的使用者。

- 攻擊者竊取「使用者使用動態 JWT Token 提出存取要求」

攻擊手法：攻擊者在使用者獲得 challenge 並產生存取要求後，竊取此要求並重送給應用程式，期望應用程式驗證此要求後，將資料回傳給攻擊者，讓攻擊者可獲取敏感資料。

防禦方法：由於此存取要求內的參數每次都會變動，透過應用程式提供的 challenge 及序號產生 JWT Token，攻擊者就算竊取存取要求並進行重送，因此時使用者已經透過此令牌進行要求，應用程式在驗證攻擊者送來的要求時，會發現已經用過，此要求會驗證失敗。每一次使用者要提出存取要求時都需要產生動態 JWT Token 進行存取要求，在驗證成功後才可進行存取，滿足 R1 需求，也就是要針對每次資料存取進行要求許可。

(2.) 中間人攻擊：在使用者與企業端中間將傳輸資料攔截下來，獲取裡面資料後在用自己的公私鑰加密，讓使用者以為自己傳給企業端，而企業端以為自己傳給使用者。

- 在使用者與身分提供者間進行中間人攻擊

攻擊手法：攻擊者對使用者假冒自己是身分提供者，同時對身分提供者假冒自己是使用者，藉由假冒身分，來假冒使用者進行設備綁定及獲取隨機值，並透過隨機值產生動態令牌，對應用程式提出要求，獲取受害者的資料。

防禦方法：在這類攻擊手法中，使用者與身分提供者皆需要確認好對方身分才可傳輸資料，否則會使得資料傳到中間人手上。攻擊者若假冒是身分提供者，讓使用者用錯誤的公鑰加密及簽章，來獲取使用者設備資料並對身分提供者假冒使用者進行綁定，但因為使用者會先確定身分提供者的憑證正確性及是否可信，才會進行後續加密及簽章，攻擊者無法偽造。而若攻擊者偽造自己是使用者，對身分提供者進行設備綁定，因身分提供者會對使用者進行身分鑑別，攻擊者亦無法偽造，整體來說，透過使用者與身分提供者雙向鑑別，讓攻擊者無法實施中間人攻擊。

- 在使用者與應用程式間進行中間人攻擊

攻擊手法：攻擊者如對使用者假冒自己是應用程式，企圖讓使用者用錯誤金鑰進行加解密而可以讓攻擊者獲取重要參數並偽造使用者對應用程式提出要求，

防禦方法：但要達成這樣的攻擊環境，需要使用者使用錯誤金鑰進行加密及簽章，而在論文預設過程中，使用者在授權應用程式存取其個人資料時，就會擁有同意過應用程式的公鑰，因此不存在使用者使用錯誤金鑰的情況發生。若攻擊者偽造使用者並對應用程式提出存取要求，希望應用程式驗證要求後，將機敏資料回傳給偽造使用者的攻擊者，但因應用程式並不是僅認存取令牌就視為此要求者為正確使用者，還會藉由身分提供者對使用者進行身分認證，因此也可避免中間人攻擊問題。



(3.) 攻擊者為使用者或身分提供者：使用者本身為攻擊者，透過存取過程竊取企業應用程式機敏資料

- 攻擊者為使用者

攻擊手法：如若攻擊者為使用者，企圖提出存取要求並獲取高權限資料，在這樣的情況下，攻擊者可直接照協定流程走，在應用程式驗證存取要求及藉由身分提供者認證使用者身分後，獲取企業內部資源。

防禦方法：不論攻擊者是否為使用者的狀態下，使用者代理都會掃描使用者設備的狀況，如果使用者設備被攻擊者入侵，可藉由設備異常行為進行判斷，給予此使用者較低的權限，讓其無法獲取企業機敏資料較高的資源。

- 攻擊者為身分提供者

攻擊手法：攻擊者為身分提供者，可直接假冒使用者並提出存取要求，在應用程式要求驗證使用者身分時，直接認證使用者身分，並讓假冒使用者的攻擊者獲取企業資源。

防禦方法：本研究主要將應用服務伺服器與身分提供者整合起來，因此在本研究提出的框架中，身分提供者本身就是可信賴的，在使用者提出登入或存取要求的過程中，會假設應用服務僅信賴身分提供者，並藉由 OAuth Token 要求身分提供者對使用者進行身分鑑別來同意使用者是否能存取。

(4.) 竄改使用者設備情況來提高存取權限：攻擊者藉由入侵使用者裝置並透過偽造來提高設備信賴程度，提升使用者存取權限，從而獲取高敏感資料

- 攻擊手法：攻擊者可透過入侵使用者設備並竄改設備狀態聲明來讓獲取高層級機密資料。

防禦方法：本研究透過使用者代理來監控使用者設備並產生設備狀態聲明，讓攻擊者無法偽造設備狀態，就無法透過此方法來獲取高敏感資料。

(5.) 使用者裝置遺失或廢止：使用者裝置遺失後，被有心人拾取後進行要求存取

- 攻擊手法：如使用者設備被攻擊者偷竊而遺失，攻擊者企圖透過使用此設備來完成使用者身分鑑別，並獲取企業內部資源。

防禦方法：在本研究中，在要進行裝置鑑別前，會先對使用者進行身分識別，即使攻擊者能夠透過裝置鑑別，也無法成功驗

證使用者身分，而導致存取失敗。即使裝置遺失導致使用者設備令牌不見無法驗證，使用者也可透過註銷流程將此裝置進行註銷，並重新綁定新裝置。

上述五點為本研究的安全性分析，透過上述的分析，可清楚了解本研究在如今新環境都需要以零信賴架構為主的時代，重新針對身分驗證進行設計，強化 token-based 驗證方法，而本研究提出的框架能夠讓既有使用 OAuth SSO 的企業能夠支援零信賴架構，以前面提出的四點要求來看，當四點要求皆符合後，在每個週期開始前，進行要求許可，而後使用者要進行要求存取時，企業端能夠清楚的驗證其身分，確保是本人才能夠讓使用者存取，支援零信賴原則內提到的應以每次連線為基礎進行許可；除此之外，在每次使用者要求存取時，企業也可針對使用者設備狀態進行分析，給予不同存取權限，也支援零信賴原則內的動態決定存取資源。針對本研究提出的框架中，能透過設備綁定及 challenge/response 的方式驗證使用者身分，並透過每次設備狀態動態決定不同存取權限來滿足四點要求，同時在設計協定的過程中，考量傳輸時的風險，並加以防範，來讓 OAuth SSO 支援零信賴架構。



第六章 結論與未來發展

本論文提出一個支持零信賴架構的 OAuth 令牌身分鑑別機制，透過此方法能讓企業不以是否在內網來作為存取控制的依據。過往企業會透過令牌來作判斷，只要持有令牌的使用者就可獲得使用者資料，但如今因遠距辦公興起，僅透過是否在內網來判斷會造成攻擊者從遠端突破邊界後，就可在內部橫向移動，獲取敏感資料。為了因應現今存取方式的改變，本研究基於零信賴架構原則強化企業存取控制，透過本研究所提出的方法，於初次使用時先做設備綁定，後續進入內網時都需要做身分識別，同時透過安裝在裝置上的程式來評估裝置風險，以便能夠基於裝置的風險去進行存取控制，這樣的方式能解決以令牌為驗證方式所造成的身分識別不足，也能動態更改每次連線的存取權限，同時也能減少企業在遠距辦公上的資安風險。

對於本研究所提出之方法，未來應可思考透過多因子認證的方式來加強驗證使用者狀況，因目前透過設備狀態來評估使用者能獲取的權限僅能初步達到權限控制，如果加入多因子認證方式後，當使用者要求企業內部敏感度最高的資料，就可要求使用者額外通過多因子認證來提高信賴程度，另外目前僅對於使用者權限做出相對應的動態改變，但日後也可針對企業內部資源的內容來做機密程度的動態調整，如分析資源內是否有個人身分識別資訊或公司商業機密等，隨時滾動式調整要存取此資源所需的信賴程度，透過資源的機密性及使用者存取權限兩者的動態改變，讓企業更能確保機敏資料及存取者的安全。

參考文獻

- [1] Hardt, D. (2012). The OAuth 2.0 authorization framework. Retrieved from <https://datatracker.ietf.org/doc/html/rfc6749>.
- [2] 王宏仁. (2019). [開放銀行特別報導]金融個資存取如何更安全?開放銀行將引進 OAuth2 委任授權架構[線上論壇]. 民 110 年 6 月 15 日，取自 <https://www.ithome.com.tw/news/133707>.
- [3] The US Centers for Medicare and Medicaid Services (CMS). (2018). “Blue button 2.0 implementation guide,” Online document. Retrieved from <https://bluebutton.cms.gov/assets/ig/index.html>.
- [4] 聯合新聞網. (2020). 遠距工作的資安風險與防護重點. 民 110 年 6 月 15 日，取自 <https://udn.com/news/story/6877/4541412>
- [5] 余至浩. (2018). 臉書驚爆史上最大漏洞攻擊，全球高達 5,000 萬用戶個資恐遭駭客竊取. 民 110 年 6 月 15 日，取自 <https://www.ithome.com.tw/news/126162>

- [6] Rose, S., Borchert, O., Mitchell, S., & Connelly, S. (2019). Zero trust architecture (No. NIST Special Publication (SP) 800-207 (Draft)). National Institute of Standards and Technology.
- [7] Wikipedia contributors. (2021). Single sign-on. Retrieved June 20, 2021, from https://en.wikipedia.org/wiki/Single_sign-on.
- [8] miniOrange. (n.d.) Single Sign On(SSO) using OAuth/OpenID Connect. Retrieved June 25,2021, from [https://www.miniorange.com/single-sign-on-\(sso\)-using-oauth](https://www.miniorange.com/single-sign-on-(sso)-using-oauth)
- [9] miniOrange. (n.d.) OAuth2.0 Server/OpenID Connect Server. Retrieved June 25,2021, from <https://www.miniorange.com/oauth2-server>
- [10] Hossain, N., Hossain, M. A., Hossain, M. Z., Sohag, M. H. I., & Rahman, S. (2018). OAuth-SSO: a framework to secure the OAuth-based SSO service for packaged web applications. In 2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE) (pp. 1575-1578). IEEE.



[11] Sakimura, Bradley, Jones, Jay. (2014). OpenID Connect Discovery 1.0 incorporating errata set 1. Retrieved from http://openid.net/specs/openid-connect-discovery-1_0.html

[12] Ward, R., & Beyer, B. (2014). Beyondcorp: A new approach to enterprise security. Retrieved June 25 , 2021, from https://www.usenix.org/system/files/login/articles/login_dec14_02_ward.pdf

[13] Gilman, E., & Barth, D. (2017). Zero Trust Networks. O'Reilly Media, Incorporated.



[14] Guide, A. E., Garbis, J., & Chapman, J. W. Zero Trust Security.

[15] Cisco. (2021) Two-Factor Authentication(2FA) from Duo. Retrieved June 26,2021, from <https://duo.com/product/multi-factor-authentication-mfa/two-factor-authentication-2fa>

[16] Common Methodology for Information Technology Security Evaluation, version 3.1 rev. 5, (2017). Retrieved June 26,2021, from <http://www.comoncriteriaportal.org/>.

[17] Dulaney, E., & Easttom, C. (2017). CompTIA Security+ Study Guide: Exam SY0-501. John Wiley & Sons.

[18] Microsoft. (n.d.) Keep your computer secure at home. Retrieved June 26, 2021, from <https://support.microsoft.com/en-us/windows/keep-your-computer-secure-at-home-c348f24f-a4f0-de5d-9e4a-e0fc156ab221>

[19] Warner, J. (2021). What is Zero Trust Security?. Retrieved June 26, 2021, from <https://www.crowdstrike.com/cybersecurity-101/zero-trust-security/>

